American International University-Bangladesh · Department of Computer Science & Engineering
CSC4161 · Advanced Programming in Web Technology

Project Report

STRESSFORGE

An architecture stress-testing REST API and web application

Group information	
Group	Individual submission (one student)
Section	D
Semester	Fall 2026–2027
Submitted to	Sultanul Arifeen Hamim, **Designation: fill in**
Submission date	**Date of submission: fill in**

Group members, contribution and AI usage

SL	Student ID	Name	Section	Contribution	AI usage
1	22-49838-3	Mubassir Ebne Mohsin	D	Requirements, database design and migrations, authentication and sessions, roles and permissions, simulation engine integration, designs and runs, reports and sharing, Support system, notifications, optional email, Admin tools, web application, tests, documentation. **Edit this to the work you personally did.**	**__ % (fill in your honest figure)**

AI usage is the share of the student's own work (code, diagrams, report text) that was drafted or generated with an AI tool, as the student's own estimate. **Confirm the tools below and your percentage before submitting.** Tools used for this project: Claude Code (Anthropic) and Codex (OpenAI) as coding assistants for implementation, tests, diagrams, screenshots and report drafting. All generated code was read, run and tested; the test results in Chapter 7 are from commands run on the final code. The student must be able to explain every module.

Contents

1. Background Study 3
1.1 Project overview and objectives 3
1.2 Problem domain 3
1.3 Existing solutions 3
1.4 Technology study 4
1.5 Key concepts 5

2. Requirement Analysis 6
2.1 Functional requirements 6
2.2 Non-functional requirements 7
2.3 Constraints and assumptions 8

3. Use Case Diagram and ER Diagram 9
3.1 Actors 9
3.2 Use case diagram 10
3.3 Use case descriptions 10
3.4 ER diagrams 12
3.5 Relationships 14
3.6 Keys and constraints 15

4. Database Normalization 18
4.1 Unnormalized form (UNF) 18
4.2 First normal form (1NF) 18
4.3 Second normal form (2NF) 18
4.4 Third normal form (3NF) 19
4.5 How the sample maps to the real tables 19
4.6 A deliberate denormalisation: the graph and the result are documents 19

5. List of APIs 21
5.1 Response format 21
5.2 Endpoints 22
5.3 Requirement coverage 26
5.4 Roles and permissions 27

6. Screenshots of Functionality 29

7. Implementation Highlights 42
7.1 A strict request contract 42
7.2 Creating a report inside a transaction 42
7.3 Permissions read from the database on every call 43
7.4 Timing-safe login 43
7.5 One exception filter for every error 43
7.6 Optional email that cannot hurt a request 43
7.7 Verification 44

8. Conclusion and References 45

1. Background Study

1.1 Project overview and objectives

STRESSFORGE is a web application and REST API for learning how a system architecture behaves under load. A person draws a design (components such as clients, load balancers, app servers, caches, databases and queues, joined by connections), sets the traffic it must survive, and runs a mathematical model that predicts latency, errors and utilisation. The result says whether the design **passes**, is **strained** or **fails**, names the bottleneck, finds the load at which the design stops meeting its target, and suggests fixes that the server checks by running the model again. Every number is a model estimate, not a measurement of real infrastructure, and the application says so on every result.

Besides the simulator the system contains what a multi-user service needs: accounts with five roles, saved designs with immutable revisions, read-only share links, a printable report, reviewer feedback, instructor challenges, a problem-report (Support) system, an in-app notification bell, optional email, and Admin tools. The project has four checkable objectives:

1. Provide REST endpoints, with the right verbs and status codes, for accounts, designs and their revisions, runs, notes, shares, reviews, challenges, Support reports and notifications (115 endpoints, Chapter 5).
2. Keep multi-table changes correct by running them inside database transactions, with row-level locking and optimistic concurrency where two people could change the same data (Chapter 7).
3. Restrict every operation by role: JWT authentication, one permission table checked by a global guard, and roles read from the database on every call (Section 5.4).
4. Validate every input at the edge with strict schemas, return one consistent error shape, and keep an append-only history of role changes and Support activity (Chapters 5 and 7).

Scope. The project delivers a NestJS backend and a Next.js web client; the guide does not require a frontend, and the API can be exercised through Swagger UI alone (Chapter 6 does exactly that). Out of scope: real traffic generation against deployed systems, production monitoring, automatic cloud provisioning and simultaneous collaborative editing.

1.2 Problem domain

An architecture diagram shows which components are connected, but not which one will be overloaded first. Students and beginning developers often add servers without finding the real constraint: a second app server does not help if the database is the bottleneck. Load-testing a real system answers the question, but needs a deployed system, scripts and money. Three problems follow:

- A diagram carries no numbers, so two designs cannot be compared objectively.
- Trying a change (more copies, a cache, a read replica) in reality means building and paying for it first.
- A result without its inputs is not reproducible: nobody can tell which traffic, which design revision and which model version produced it.

STRESSFORGE addresses these with a deterministic model that anyone can re-run, saves every design change as an immutable revision, stores the inputs of every run with its result, and marks a result **out of date** when the design has changed since (a SHA-256 fingerprint of the inputs is compared).

1.3 Existing solutions

The comparison is by category of tool, because the point is what each approach can and cannot answer for a learner.

Approach	Strengths	Gaps for this purpose
Diagramming tools (for example draw.io, Excalidraw)	Free-form drawing, widely known, easy sharing	No load model: a diagram never says what will break first
Load-testing tools (for example k6, Apache JMeter)	Real measurements of a real system	Need a running, deployed system and test scripts; cost and set-up time; not usable at design time
Cloud pricing calculators and architecture centres	Cost estimates and reference diagrams	Cost, not latency or failure behaviour under a given traffic
Textbook queueing formulas by hand	Exact, explainable	Slow to apply to a whole graph, no persistence, no comparison between versions
STRESSFORGE (this project)	Draw, set traffic, get a predicted verdict, bottleneck and verified fixes in seconds; every run is saved with its inputs	A model, not a measurement; no real traffic; no real cloud provisioning

1.4 Technology study

Versions are taken from the `package.json` files of the two packages (backend and web) at the time of writing.

Technology	Role in STRESSFORGE	Why it was chosen
NestJS 11.2.7 (TypeScript 5.9.3, Node.js 22 or later)	Backend framework	Modules, dependency injection, guards, pipes, interceptors and filters are built in and map directly onto the course topics
PostgreSQL 16	Relational database, the system of record	ACID transactions, row locking, advisory locks, CHECK and unique constraints, triggers, jsonb for frozen documents
TypeORM 0.3.31 with pg 8.23.1	ORM, migrations and driver	Entities, migrations and a DataSource with transactions; most business queries are parameterised SQL inside transactions for exact control
Redis 7 (ioredis 5.11.1, redis 5.12.1)	Rate-limit counters, token revocation, AI quota	Fast shared counters that survive restarts and work across instances
Bull 4.16.5 (@nestjs/bull 11.0.5)	Job queue for the optional email outbox	Redis-backed repeating jobs with retries
JWT (@nestjs/jwt 11.0.2) and argon2 0.44.0	Access tokens and password hashing	Stateless short-lived access tokens; argon2 is a slow, memory-hard password hash
Zod 4.6.7 with nestjs-zod 5.5.0	Validation of every request, response and engine input	One strict schema is both the runtime check and the TypeScript type; unknown fields are rejected
@nestjs/throttler 6.7.1	Rate limiting	Per-client limits with Redis storage
@nestjs/swagger 11.4.7 (OpenAPI 3.1)	Interactive API documentation at /docs	Generated from the schemas, with bearer authentication for the demo
Helmet 8.3.0, cookie-parser 1.4.7	Security headers, cookie reading	Standard Express hardening
nodemailer 10.4.0	SMTP client for the optional email feature	Small and well known; Mailtrap sandbox for development
Node worker_threads	Runs the simulation engine off the main thread	A heavy graph cannot block other requests; runs have a hard time limit
Next.js 16.4.2, React 19.3.0	Web client	File-based routes, production build
TanStack React Query 5.104.5	Server-state cache in the web client	Caching, refetching, polling of runs and notifications
pdf-lib 1.17.1	PDF report built in the browser	Pure JavaScript, no server round trip
Jest 30.5.2, supertest 7.3.1	Backend unit and end-to-end tests	End-to-end tests run against a real PostgreSQL test database
Playwright 1.64.1	Browser journeys against a production build	Real clicks in Chrome, light and dark themes

Technology	Role in STRESSFORGE	Why it was chosen
Docker Compose	Starts PostgreSQL and Redis for development	Same services on every machine

1.5 Key concepts

Authentication and sessions

1. `POST /v1/auth/login` checks the email and password (argon2 verify) and returns a short-lived **access token** (JWT, 900 seconds by default) in the response body; the browser keeps it in memory only.
2. The same response sets a **refresh token** in an httpOnly cookie named `sf_refresh`, restricted to the path `/v1/auth`, `SameSite=Lax` and `Secure` in production, so page scripts cannot read it and the rest of the API never receives it.
3. Every request sends `Authorization: Bearer <access token>`. The global `AuthGuard` verifies the signature and expiry and checks Redis for a revoked token id, a blocked user, or a session ended by a password change. A missing or bad token gives **401**.
4. When the access token expires the client calls `POST /v1/auth/refresh`. The refresh token is **rotated**: each use returns a new one. Presenting an already-used refresh token is treated as theft and revokes the whole token family.
5. Permissions are **not** taken from the token. The `PermissionGuard` reads the account's roles from the `user_roles` table on every protected call, so a revoked role stops working at once. A missing permission gives **403**.

Why transactions matter here

Creating a Support report changes several tables: the ticket row, the activity log row, and the notifications for Support staff. If the notification insert failed after the ticket had been saved, a report would exist that nobody was told about. A transaction makes the group atomic: all changes are saved, or none. Section 7.2 shows the code, and Figure 6.11 shows a test that forces a failure in the middle of a transaction and proves nothing was left behind.

Optimistic concurrency and idempotency

Two browser tabs can edit the same design. Each save names the revision it started from (`expectedRevision`); if the stored revision moved on, the server answers **409 Conflict** with the current revision instead of overwriting the other tab's work (Figure 6.8). Run requests carry a `requestId`, so a double click or a network retry returns the first run instead of starting a second one.

2. Requirement Analysis

2.1 Functional requirements

Each requirement names the module that implements it. Priority is High for the core loop, Medium for supporting features and Low for convenience.

ID	Requirement	Priority	Module
Accounts and sessions			
FR-01	A visitor can register with an email, a display name and a password of 8 to 128 characters with at least one letter and one digit. The email is stored in lower case and is unique. New accounts have the User role.	High	Auth
FR-02	A user can log in and receives a short-lived access token and a refresh cookie.	High	Auth
FR-03	The refresh token is rotated on every use; reuse of an old one revokes the whole token family.	High	Auth
FR-04	A user can log out; changing the password ends every earlier session.	High	Auth
FR-05	A suspended account cannot sign in and its tokens stop working.	High	Roles
FR-06	A signed-in user can read their own profile and roles, and change their display name.	Medium	Auth, Account
FR-07	A user can export everything they own as JSON and delete their own account after typing a confirmation. Their designs are removed; their Support reports and challenge feedback are kept anonymised.	Medium	Account
Designs and simulation			
FR-08	A user can create a design from a blank canvas, one of six templates, a validated import (at most 65,536 bytes) or a text description.	High	Designs
FR-09	A design has at most 50 components of 12 types and at most 200 connections; the server validates the graph and reports each problem with a code.	High	Designs, Simulation
FR-10	A save names the revision it started from; a stale save is refused with 409 and the current revision.	High	Designs
FR-11	Every save creates an immutable version. A user can list versions, compare two, and restore an old one (which creates a new version).	High	Designs
FR-12	A user can list own designs with paging, search, filters and sorting, and can duplicate, archive, unarchive and delete a design.	Medium	Designs
FR-13	A user can attach notes of kind decision or assumption (1 to 2,000 characters) to a design, optionally pinned to a component.	Medium	Designs
FR-14	A user can run a stress test. The call is asynchronous (202 Accepted); the user polls the run and gets a verdict (pass, strained, fail), p50, p95 and p99 latency, error rate, per-component utilisation and the bottleneck.	High	Designs, Simulation
FR-15	A run request carries a `requestId`; repeating it returns the same run.	High	Designs
FR-16	A user can search for the breaking load (Limits), run a load ladder at 2x, 5x and 10x, ask a what-if question, and get up to three fix suggestions that the server has verified by re-running the model.	High	Designs, Simulation
FR-17	A user can preview an unsaved graph live; previews have their own rate-limit bucket.	Medium	Simulation
Reports, sharing and review			
FR-18	A user can open a report for a design (a consistent snapshot of the design, one revision and its latest run) and download it as a PDF built in the browser.	High	Designs, Web
FR-19	A user can create a read-only share link (the token is shown once and only its hash is stored) that expires after 7, 30 (default) or 90 days or never, list links and revoke a link. Anyone with a valid link can open the shared view without signing in.	High	Designs
FR-20	A design owner can grant one named Reviewer access to one revision, revoke it, and read and answer the comment thread.	Medium	Reviews
FR-21	A Reviewer can list designs shared with them, read the shared revision and add, edit and remove their own comments.	Medium	Reviews

ID	Requirement	Priority	Module
Learning			
FR-22	Anyone can list and read published challenges. A signed-in user can start an attempt, submit it (202 Accepted) and see their progress.	Medium	Challenges
FR-23	An Instructor can create, edit, publish, archive and duplicate challenges, read submissions to them and give feedback.	Medium	Challenges
Support, notifications and email			
FR-24	A user can send a problem report (category, title 3 to 120 and body 10 to 4,000 characters, an allow-listed technical context). At most 5 reports an hour and 20 a day per account; an open report with the same title is refused with 409 unless the user confirms.	High	Support
FR-25	A user can list their own reports, read their thread (internal notes are never shown) and reply; a resolved report can be continued for 14 days.	High	Support
FR-26	Support staff can list and search the inbox, read a report, mark it read, reply, add internal notes, set status and priority, assign it, use saved replies (personal or team) and merge or unmerge duplicate reports.	High	Support
FR-27	A user sees notifications for Support replies and status changes (staff also for new reports, assignments and reporter replies), can mark them read, and can mute each kind.	Medium	Notifications
FR-28	When enabled, the reporter also receives an email for a Support reply or a status change. The email never contains report text.	Medium	Mails
Administration			
FR-29	An Admin can search accounts, grant and revoke the Instructor, Reviewer, Support and Admin roles, and suspend and restore accounts. The last Admin cannot be removed or suspended and nobody can suspend themselves.	High	Roles
FR-30	Every role change and suspension is written to an append-only audit log that an Admin can read.	High	Roles
FR-31	An Admin can view a live dashboard of aggregate counts and export a snapshot as CSV or JSON. The dashboard never reads design content or the Support inbox.	Medium	Roles
Platform			
FR-32	All successful responses use the envelope `{ data }` (lists add `meta`), all failures use `{ error: { code, message, details } }`, and lists are paged.	High	Common
FR-33	Optional AI features (describe a system in words, what-if sentence, fix filter) are off by default and only fill a strict form; all numbers and graphs come from the engine.	Low	AI
FR-34	A public health endpoint reports whether PostgreSQL and Redis answer.	Medium	Health

2.2 Non-functional requirements

Each target is a number that can be checked. The last column says how.

ID	Category	Requirement	How it is checked
NFR-01	Security	Passwords are stored only as argon2 hashes; a login for an unknown email still runs a dummy hash so timing does not reveal which emails exist. JWT secrets and lifetimes come from the environment (access token 900 s, refresh token 7 days by default).	Inspect the users table and the configuration; unit tests of the auth service
NFR-02	Security	Every route except 8 public ones requires a valid token. 39 routes also require a role permission; roles are read from the database on every call.	Probe of all 115 routes (Section 5.2): 8 answered without a token, 39 answered 403 to an ordinary user
NFR-03	Security	Request bodies are limited to 100 kb (160 kb for design import); text fields have maximum lengths; unknown fields are rejected with 422.	Swagger 422 test (Figure 6.7); contract tests

ID	Category	Requirement	How it is checked
NFR-04	Performance	A saved run is stopped after 5 seconds and a live preview after 2 seconds; at most 4 runs wait in the admission queue.	Executor settings in `simulation-executor.ts`; worker tests
NFR-05	Rate limiting	120 requests a minute per client by default, 10 a minute for register, login and refresh, 5 problem reports an hour and 20 a day per account.	Throttler configuration; Support end-to-end tests
NFR-06	Data integrity	Multi-table writes run in transactions. The schema has 34 foreign keys and 63 CHECK constraints, plus triggers that make the audit tables append-only.	Schema dump (Chapter 3); forced-failure test (Figure 6.11)
NFR-07	Concurrency	A design save names its revision and a stale save is refused with 409; concurrent writes to the same record (saved-reply edits, merges) cannot both succeed; two simultaneous requests cannot both exceed a per-account limit.	Figure 6.8; concurrency tests in `support-extras.e2e-spec.ts` (simultaneous merges, edits and creates)
NFR-08	Privacy	Admin cannot read design content or the Support inbox; internal Support notes are never returned to the reporter; emails contain no report text.	Permission table; end-to-end tests in the roles, support and mails specs
NFR-09	Auditability	Role changes and Support activity are stored in append-only tables; an UPDATE or DELETE on them fails in the database.	Triggers `role_audit_no_change` and `support_activity_no_change`; tests
NFR-10	Usability	Every endpoint is documented in Swagger (115 operations). An invalid request names each failing field.	Swagger review (Chapter 6); 422 body in Figure 6.7
NFR-11	Maintainability	One NestJS module per feature, one strict schema per operation; lint and type checks pass.	ESLint and `tsc` on the changed files; 63 unit-test suites and 33 end-to-end suites
NFR-12	Portability	The schema is built only from migrations (21 of them) and every setting comes from environment variables validated at start-up.	Migrations on an empty database (Figure 6.15)
NFR-13	Scalability	The API keeps no session state in memory: tokens are self-contained, counters and revocations live in Redis, data lives in PostgreSQL. The simulation admission queue is per instance.	Design review

2.3 Constraints and assumptions

- The model is a mathematical estimate. It is never presented as a measurement of real infrastructure.
- PostgreSQL and Redis run locally through Docker Compose; no paid cloud service is needed.
- Email is optional, off by default and sent only to a Mailtrap sandbox inbox during development and the demo.
- AI features are optional and off by default; the simulator remains the only source of numbers.
- The scope is a single organisation: one database, no multi-tenant isolation.

3. Use Case Diagram and ER Diagram

3.1 Actors

Actor	Description	Main permissions
User	Every account (the implicit base role)	Create and run own designs, notes, shares, reports and PDFs; send problem reports; read notifications; take challenges; export or delete the account
Instructor	Granted by an Admin	Create, edit, publish, archive and duplicate challenges; read submissions to them; give feedback
Reviewer	Granted by an Admin	Read designs that owners shared with them (one frozen revision) and comment
Support	Granted by an Admin	Read and answer problem reports, write internal notes, set status and priority, assign, saved replies, merge duplicates. Never sees designs
Admin	First one created by a server command, others by an Admin	Manage accounts and roles, suspend and restore, read the audit log, see aggregate dashboard numbers. Never reads design content or the Support inbox
Email service	Secondary actor (SMTP server, optional)	Receives the mails the system sends to reporters
AI provider	Secondary actor (hosted model, optional)	Fills a strict form for three narrow tasks; never produces numbers

Roles are **not inherited**: an Admin does not gain Instructor, Reviewer or Support powers, and no role reads another person's private design. Section 5.4 gives the permission table.

3.2 Use case diagram

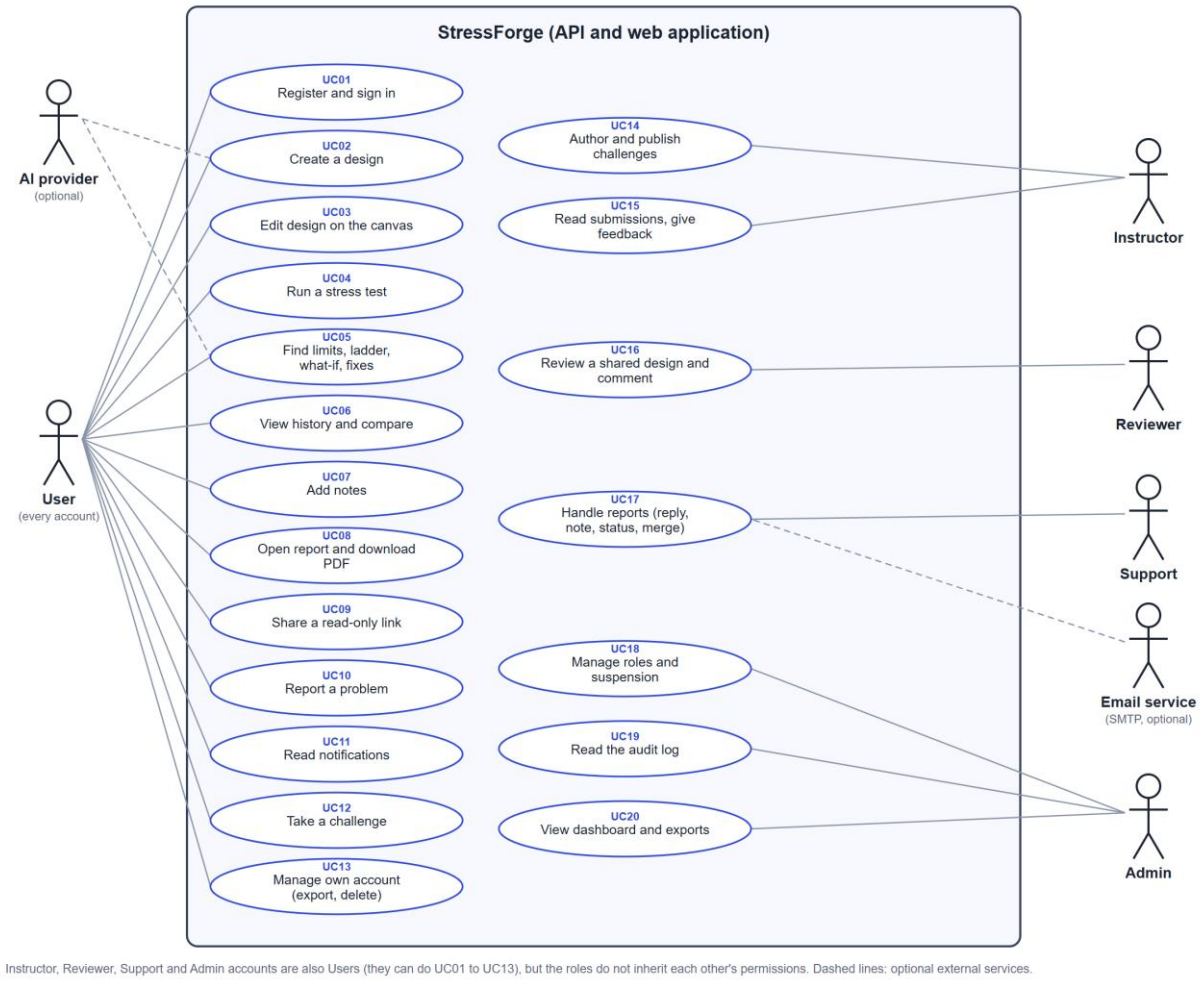


Figure 3.1: Use case diagram of STRESSFORGE

3.3 Use case descriptions

UC04 Run a stress test

Field	Description
Actor	User (primary)
Preconditions	The user is signed in and owns the design. The design has a saved revision.
Main flow	1. The client sends `POST /v1/designs/{id}/simulate` with the revision it believes is saved (`expectedRevision`) and a fresh `requestId`. 2. The server checks ownership. If the same `requestId` was already accepted, it returns that run instead of starting another. 3. It checks that the revision is still current and that the account has no other calculation pending. 4. It validates the graph (components, connections, entry point, cycles, unreachable parts) and reserves a slot in the executor. 5. In one transaction it locks the design row, re-checks the revision and inserts a `design_runs` row with status pending, the input fingerprint and the engine version. 6. It answers **202 Accepted** with a status URL; the model runs in a worker thread with a 5 second limit. 7. The client polls `GET /v1/designs/{id}/runs/{runId}` until the status is completed or failed, then shows the verdict, percentiles, utilisation and bottleneck.

Field	Description
Alternative flows	A1. The revision is stale: **409** with `REVISION_MISMATCH` and the current revision. A2. The same `requestId` is reused for different inputs: **409** with `REQUEST_ID_REUSED`. A3. The graph is invalid: **422** with the list of problems. A4. Another calculation is pending for the account, or capacity is exhausted: **429** (retry with the same `requestId`). A5. The calculation service is unavailable or the admission window expired: **503**. A6. The server restarts while a run is pending: the run is marked failed (interrupted) at the next recovery pass.
Postcondition	Either one run row exists with its inputs and (when finished) its result, or nothing was stored.

UC10 and UC17 Report a problem and receive a reply

Field	Description
Actors	User (reporter), Support (staff), Email service (secondary)
Preconditions	The reporter is signed in. A Support account exists.
Main flow	1. The reporter sends `POST /v1/support/reports` (category, title, body, optional expected result, allow-listed context). 2. The server validates the body against the strict schema and, in one transaction, takes a per-account advisory lock, counts the account's reports (limits 5 an hour and 20 a day), refuses an open report with the same title unless confirmed, inserts the ticket, writes the activity row and creates the notifications for Support staff. It answers **201**. 3. A Support agent lists the inbox, opens the report and replies (`POST /v1/support/inbox/{id}/messages`). 4. The server stores the message, writes the activity row and creates a `support_reply` notification for the reporter in the same transaction. 5. The reporter's bell shows the notification. If email is enabled, a background job later sends a short email (report number and a link, never the text of the report) and records the outcome in `notification_emails`.
Alternative flows	A1. More than 5 reports in an hour or 20 in a day: **429** with `retryAfterSeconds`. A2. An open report with the same title exists: **409** `DUPLICATE_OPEN`, unless the request sets `confirmDuplicate`. A3. The reporter opens someone else's report id: **404** (another person's report looks the same as a missing one). A4. A staff-only route is called by an ordinary user: **403**. A5. The mail server is unavailable: the email is retried up to three times and then marked failed; the Support action is never affected.
Postcondition	The ticket, its activity row and the notifications exist together, or none of them do.

UC18 Manage roles and suspension

Field	Description
Actor	Admin
Preconditions	The Admin is signed in and holds the Admin role in the database.
Main flow	1. The Admin searches an account (`GET /v1/admin/users?q=`). 2. The Admin grants a role (`POST /v1/admin/users/{id}/roles`) or suspends the account. 3. In one transaction the server writes `user_roles` (or the suspension) and appends a row to `role_audit`. 4. The next request from that account sees the new roles, because roles are read from the database on every call.
Alternative flows	A1. The caller lacks the permission: **403**. A2. The last Admin would be removed or suspended, or an Admin suspends themselves: **409** (`LAST_ADMIN` or `SELF`). A3. The target account does not exist: **404**.

Field	Description
Postcondition	The role change and its audit row exist together. The audit table cannot be edited or deleted from (database trigger).

3.4 ER diagrams

The database has 24 application tables (plus the migrations bookkeeping table), 34 foreign keys, 63 CHECK constraints and 3 triggers. The diagrams are drawn from the schema of a database built by running all 21 migrations on an empty database, so keys and cardinalities match the real schema. Crow's foot notation: a bar is "one", a circle is "optional", a fork is "many". Primary keys are marked PK and foreign keys FK; "(null)" marks a nullable column. Only the important columns are shown.

ER diagram 1: accounts, designs, versions, runs, notes and share links

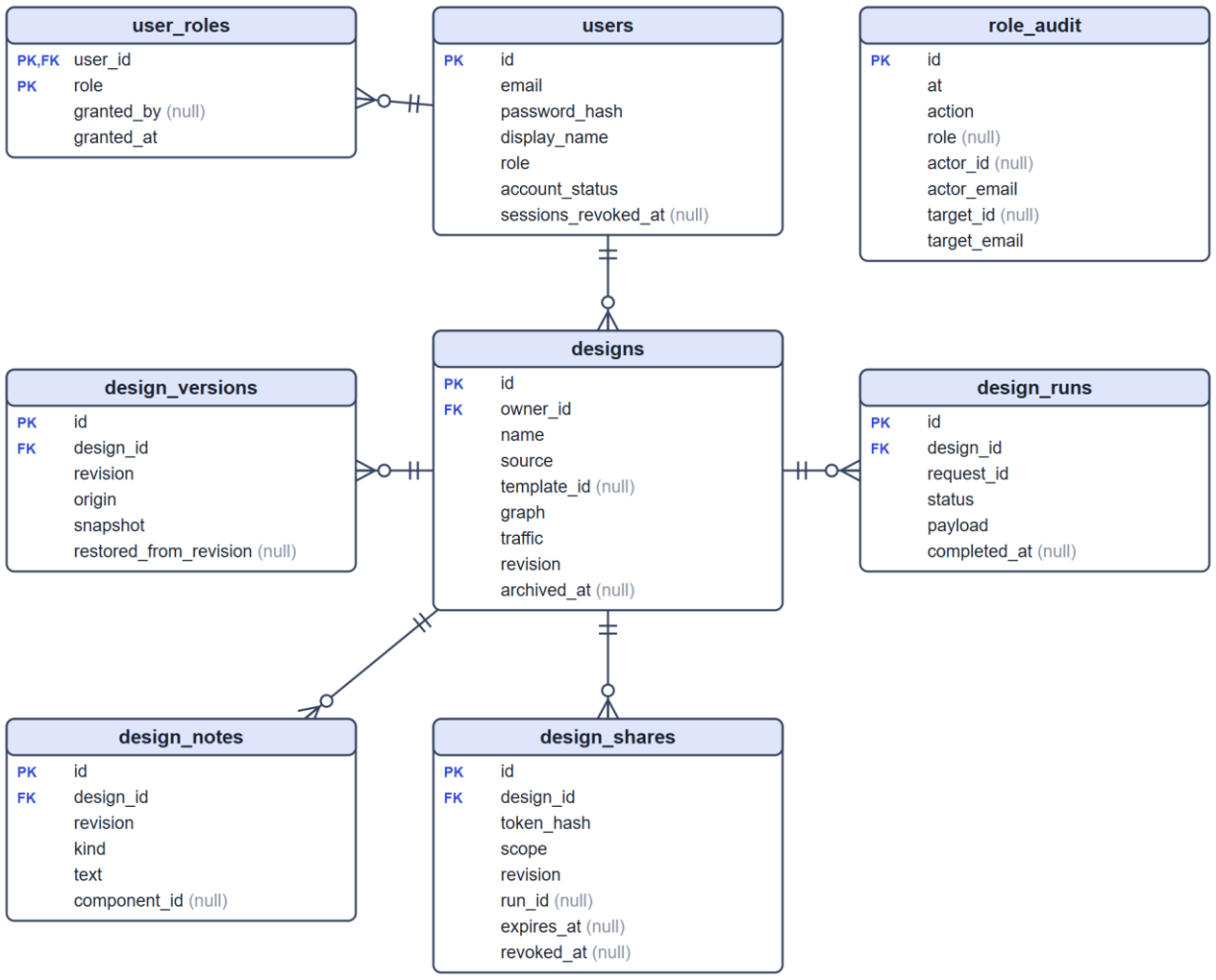


Figure 3.2: ER diagram 1, accounts, roles, designs, versions, runs, notes and share links

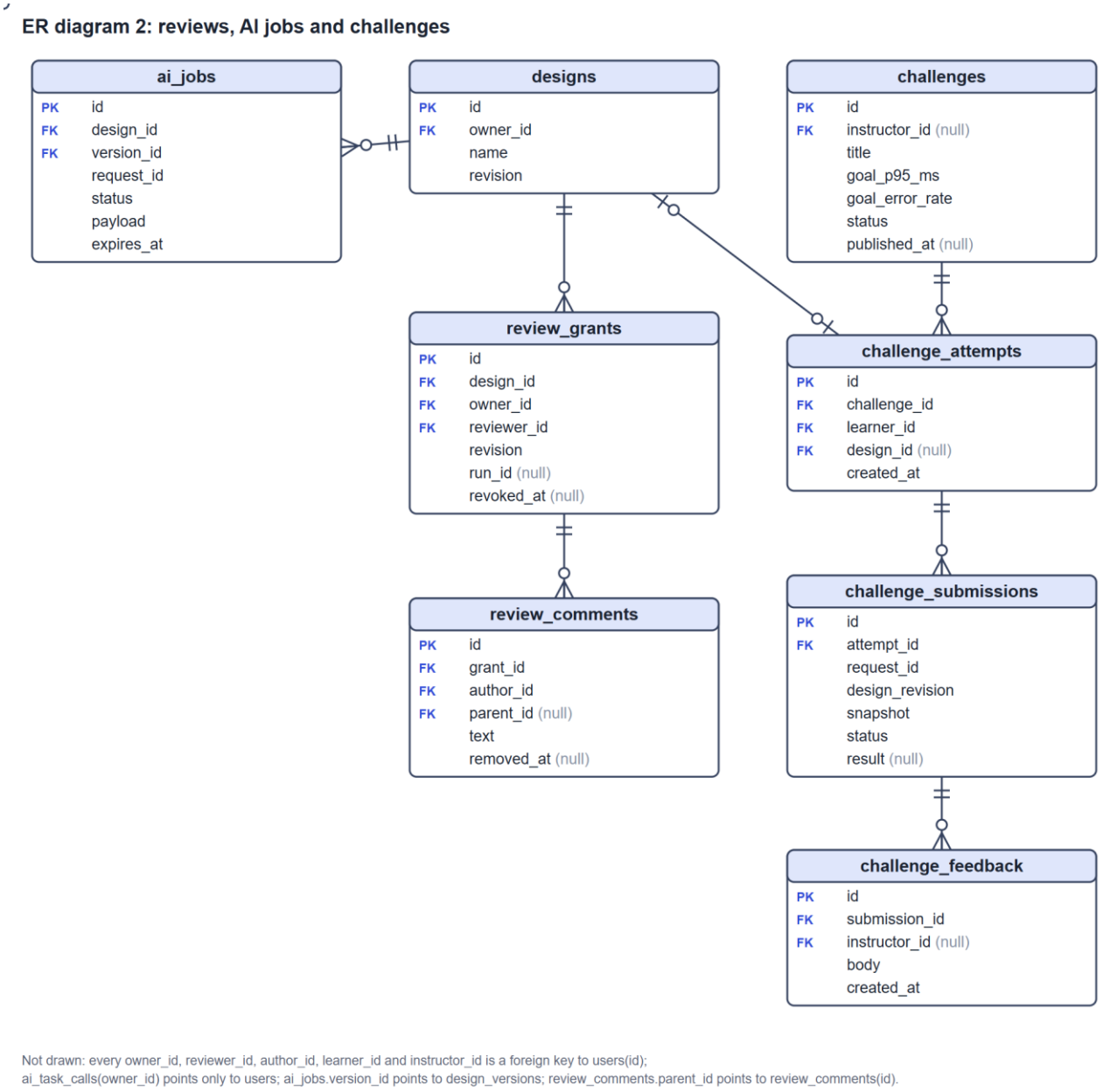


Figure 3.3: ER diagram 2, review grants and comments, AI jobs and challenges

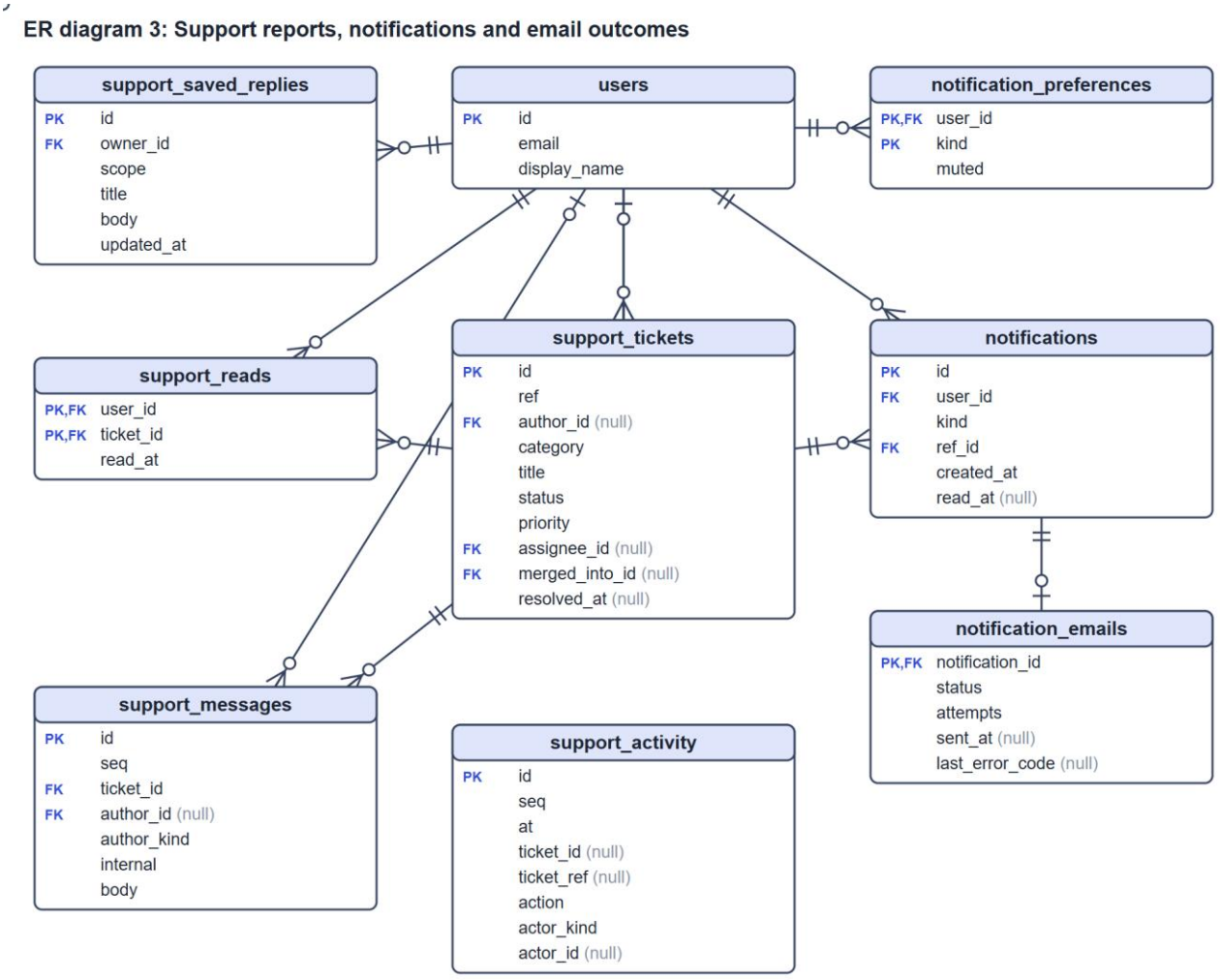


Figure 3.4: ER diagram 3, Support reports, notifications and email outcomes

> **Two tables have no foreign keys on purpose.** `role_audit` and `support_activity` are append-only histories that must outlive the accounts and tickets they describe, so they store ids and e-mail text without a foreign key. A database trigger refuses UPDATE and DELETE on both.

3.5 Relationships

Generated from the 34 foreign keys. Cardinality reads parent : child; `0..1` on the parent side means the foreign key column may be null. **On delete** is what the database does to child rows when the parent row is deleted: CASCADE removes them, SET NULL keeps them anonymised, RESTRICT or NO ACTION refuses the deletion.

Relationship	Cardinality	Implemented by	On delete
Challenge attempt has submissions	1 : 0..N	`challenge_submissions.attempt_id`	CASCADE
Submission has feedback entries	1 : 0..N	`challenge_feedback.submission_id`	CASCADE
Challenge has attempts	1 : 0..N	`challenge_attempts.challenge_id`	RESTRICT
Design version has AI jobs	1 : 0..N	`ai_jobs.version_id`	NO ACTION
Design has AI jobs	1 : 0..N	`ai_jobs.design_id`	CASCADE
Design has attempts	0..1 : 0..1	`challenge_attempts.design_id`	SET NULL
Design has notes	1 : 0..N	`design_notes.design_id`	CASCADE
Design has runs	1 : 0..N	`design_runs.design_id`	CASCADE
Design has share links	1 : 0..N	`design_shares.design_id`	CASCADE

Relationship	Cardinality	Implemented by	On delete
Design has versions	1 : 0..N	`design_versions.design_id`	CASCADE
Design has review grants	1 : 0..N	`review_grants.design_id`	CASCADE
Notification has email outcomes	1 : 0..1	`notification_emails.notification_id`	CASCADE
Review comment has replies comments	0..1 : 0..N	`review_comments.parent_id`	CASCADE
Review grant has comments	1 : 0..N	`review_comments.grant_id`	CASCADE
Support report has notifications	1 : 0..N	`notifications.ref_id`	CASCADE
Support report has messages	1 : 0..N	`support_messages.ticket_id`	CASCADE
Support report has read marks	1 : 0..N	`support_reads.ticket_id`	CASCADE
Support report has replies Support reports	0..1 : 0..N	`support_tickets.merged_into_id`	SET NULL
User has AI task calls	1 : 0..N	`ai_task_calls.owner_id`	CASCADE
User has attempts	1 : 0..N	`challenge_attempts.learner_id`	CASCADE
User has feedback entries	0..1 : 0..N	`challenge_feedback.instructor_id`	SET NULL
User has challenges	0..1 : 0..N	`challenges.instructor_id`	CASCADE
User has designs	1 : 0..N	`designs.owner_id`	CASCADE
User has preferences	1 : 0..N	`notification_preferences.user_id`	CASCADE
User has notifications	1 : 0..N	`notifications.user_id`	CASCADE
User has comments	1 : 0..N	`review_comments.author_id`	CASCADE
User has review grants	1 : 0..N	`review_grants.owner_id`	CASCADE
User has review grants	1 : 0..N	`review_grants.reviewer_id`	CASCADE
User has messages	0..1 : 0..N	`support_messages.author_id`	SET NULL
User has read marks	1 : 0..N	`support_reads.user_id`	CASCADE
User has saved replies	1 : 0..N	`support_saved_replies.owner_id`	CASCADE
User has Support reports	0..1 : 0..N	`support_tickets.assignee_id`	SET NULL
User has Support reports	0..1 : 0..N	`support_tickets.author_id`	SET NULL
User has role assignments	1 : 0..N	`user_roles.user_id`	CASCADE

3.6 Keys and constraints

Primary keys, unique constraints and CHECK constraints per table, taken from the same schema dump. CHECK constraints are the database's own validation, a second wall behind the request schemas.

Table	Constraints
`ai_jobs`	**PK** id **Unique** design_id + request_id **Check** status IN (pending, completed, failed, cancelled, interrupted)
`ai_task_calls`	**PK** id **Check** completion_tokens >= 0; input_hash ~ '^[a-f0-9]{64}$'; input_length >= 0; latency_ms >= 0; prompt_tokens >= 0; task IN (describe, whatif, fixes); total_tokens >= 0
`challenge_attempts`	**PK** id **Unique** challenge_id + learner_id; design_id
`challenge_feedback`	**PK** id **Check** (char_length(body) >= 1) AND (char_length(body) <= 2000)
`challenge_submissions`	**PK** id **Unique** attempt_id + request_id **Check** design_revision > 0; status IN (pending, completed, failed, interrupted)

Table	Constraints
challenges	**PK** id **Check** ((status = 'draft') AND (published_at IS NULL) AND (archived_at IS NULL)) OR ((status = 'published') AND (published_at IS NOT NULL) AND (archived_at IS NULL)) OR ((status = 'archived') AND (archived_at IS NOT NULL)); (goal_error_rate >= 0) AND (goal_error_rate <= 1); (goal_p95_ms >= 1) AND (goal_p95_ms <= 60000); status IN (draft, published, archived); char_length(summary) >= 1; char_length(title) >= 1
design_notes	**PK** id **Check** kind IN (decision, assumption); revision > 0; (char_length(text) >= 1) AND (char_length(text) <= 2000)
design_runs	**PK** id **Unique** design_id + request_id **Check** status IN (pending, completed, failed)
design_shares	**PK** id **Check** open_count >= 0; revision > 0; scope IN (design, design_report); char_length(token_hash) = 64
design_versions	**PK** id **Unique** design_id + revision **Check** origin IN (baseline, create, edit, restore); restored_from_revision > 0; revision > 0
designs	**PK** id **Check** revision > 0; schema_version = 1; source IN (blank, template, import, scripted, ai)
notification_emails	**PK** notification_id **Check** (attempts >= 0) AND (attempts <= 10); status IN (retry, sent, failed, skipped)
notification_preferences	**PK** user_id, kind **Check** kind IN (support_reply, report_status, support_new_report, support_assigned, reporter_reply)
notifications	**PK** id **Check** kind IN (support_reply, report_status, support_new_report, support_assigned, reporter_reply)
review_comments	**PK** id **Check** ((removed_at IS NULL) AND (char_length(text) >= 1)) OR ((removed_at IS NOT NULL) AND (text = ''))
review_grants	**PK** id **Check** owner_id <> reviewer_id; revision > 0
role_audit	**PK** id **Check** action IN (grant, revoke, bootstrap, suspend, restore); role IN (instructor, reviewer, support, admin)
support_activity	**PK** id **Check** action IN (create, reply, note, status, priority, assign, reopen, merge, unmerge, saved_create, saved_update, saved_delete); actor_kind IN (user, support); ((ticket_id IS NOT NULL) AND (ticket_ref IS NOT NULL)) OR action IN (saved_create, saved_update, saved_delete)
support_messages	**PK** id **Check** author_kind IN (user, support); (char_length(body) >= 1) AND (char_length(body) <= 4000); (NOT internal) OR (author_kind = 'support')
support_reads	**PK** user_id, ticket_id
support_saved_replies	**PK** id **Check** (char_length(body) >= 1) AND (char_length(body) <= 4000); scope IN (personal, team); (char_length(title) >= 3) AND (char_length(title) <= 120)
support_tickets	**PK** id **Unique** ref **Check** (merged_into_id IS NULL) OR (status = 'resolved'); (merged_into_id IS NULL) OR (merged_into_id <> id); (char_length(body) >= 10) AND (char_length(body) <= 4000); category IN (bug, wrong_result, account, performance, idea, other); (status = 'resolved') = (resolved_at IS NOT NULL); jsonb_typeof(context) = 'object'; (expected IS NULL) OR (char_length(expected) <= 2000); priority IN (low, normal, high, urgent); status IN (open, in_progress, waiting, resolved); char_length(title) >= 3
user_roles	**PK** user_id, role **Check** role IN (instructor, reviewer, support, admin)
users	**PK** id **Unique** email **Check** email = lower(email); role IN (user, admin); account_status IN (active, suspended)

Triggers and indexes of note

Item	Purpose
Trigger role_audit_no_change	Refuses UPDATE and DELETE on role_audit (append-only history)

Item	Purpose
Trigger `support_activity_no_change`	Refuses UPDATE and DELETE on `support_activity`
Trigger `support_merge_guard`	Prevents a report from being merged into itself, into a merged report or in a cycle
Unique `uq_users_email`, check `ck_users_email_lowercase`	One account per email, stored in lower case
Unique `design_versions (design_id, revision)`	One immutable version per revision number
Unique `design_runs (design_id, request_id)`	Idempotency of run requests
Unique `challenge_attempts (challenge_id, learner_id)`	One attempt per learner per challenge
Partial unique index `ai_jobs_one_pending`	At most one pending AI job at a time
67 indexes in total	Owner lists, cursor paging on (created_at, id), idempotency keys and partial indexes for open reports

4. Database Normalization

Before the application existed, run results could have been kept in a sheet like the one below: one row per test run, with the owner and the design copied into every row. The two sample rows and the table below are **invented for this explanation**; the normalised relations at the end correspond to real tables.

4.1 Unnormalized form (UNF)

Run	Owner	Owner e-mail	Design	Components	RPS	Verdict	Bottleneck	Notes
R-101	U-07 Nadia Karim	nadia@mail.com	D-3 Shop MVP	Users; App servers; Postgres	300	pass	none	baseline
R-102	U-07 Nadia Karim	nadia@mail.com	D-3 Shop MVP	Users; App servers; Postgres	900	fail	Postgres	decision: keep one database; assumption: reads are 80%
R-103	U-09 Imran Hossain	imran@mail.com	D-5 Blog	Users; CDN; App servers	500	pass	none	

Problems: the Components and Notes cells hold several values; the owner's name and e-mail are repeated on every run of the same person; the design name is repeated on every run of the same design; and the Bottleneck cell is meaningless for a passing run.

4.2 First normal form (1NF)

1NF requires atomic values and no repeating groups. Each component gets its own row, so the row is identified by the composite key (Run, Component). Notes move to their own relation, keyed by (Design, NoteNo).

Run	Component	Owner	Owner e-mail	Design	Design name	RPS	Verdict	Bottleneck
R-101	Users	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	300	pass	none
R-101	App servers	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	300	pass	none
R-101	Postgres	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	300	pass	none
R-102	Users	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	900	fail	Postgres
R-102	App servers	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	900	fail	Postgres
R-102	Postgres	U-07 Nadia Karim	nadia@mail.com	D-3	Shop MVP	900	fail	Postgres
R-103	Users	U-09 Imran Hossain	imran@mail.com	D-5	Blog	500	pass	none
R-103	CDN	U-09 Imran Hossain	imran@mail.com	D-5	Blog	500	pass	none
R-103	App servers	U-09 Imran Hossain	imran@mail.com	D-5	Blog	500	pass	none

Primary key: (Run, Component). Notes: `design_note(Design, NoteNo, Kind, Text)` with primary key (Design, NoteNo).

4.3 Second normal form (2NF)

2NF removes partial dependencies: a non-key column that depends on only part of a composite key. Every attribute except Component (owner, design, RPS, verdict, bottleneck) depends on **Run** alone, not on Component. Those columns move to a relation keyed by Run, and the link between runs and components stays on its own.

```
run(Run PK, Owner, OwnerEmail, Design, DesignName, RPS, Verdict, Bottleneck)
run_component(Run FK, Component)            PK (Run, Component)
```

Every non-key column in `run` now depends on the whole key. Redundancy remains: the owner's details are still repeated for every run of that person, and the design name for every run of that design.

4.4 Third normal form (3NF)

3NF removes transitive dependencies: a non-key column that depends on another non-key column. Three appear in `run`:

- Run → Design → DesignName, Owner: the design's name and owner belong to the design, not to the run.
- Design → Owner → OwnerName, OwnerEmail (through the owner): the person's name and e-mail belong to the person.
- Verdict and Bottleneck describe one run's result; they stay with the run, but their exact numbers (percentiles, utilisation per component) are a variable-size result, which is kept as a single stored document (see 4.6).

Resulting relations:

```
owner(Owner PK, Name, Email UNIQUE)
design(Design PK, Owner FK, Name)
run(Run PK, Design FK, RPS, Verdict, Bottleneck)
run_component(Run FK, Component)            PK (Run, Component)
design_note(Design FK, NoteNo, Kind, Text)   PK (Design, NoteNo)
```

In each relation every determinant is a candidate key (`Owner`, `Email`, `Design`, `Run`, or the composite keys of the two link relations), so the design is also in Boyce-Codd normal form.

4.5 How the sample maps to the real tables

Sample relation	Real table	Key columns	Notes
owner	`users`	`id` PK, `email` UNIQUE, `display_name`, `password_hash`	Email stored lower-case (CHECK); roles split into `user_roles`
design	`designs`	`id` PK, `owner_id` FK to `users`	Holds the current draft graph and traffic as jsonb, plus `revision`
design versions	`design_versions`	`id` PK, UNIQUE (`design_id`, `revision`)	Immutable snapshot of each saved revision
run	`design_runs`	`id` PK, `design_id` FK, UNIQUE (`design_id`, `request_id`)	Status columns plus the result in `payload`
design_note	`design_notes`	`id` PK, `design_id` FK	Kind limited to decision or assumption; optional `component_id`
run_component	inside `design_versions.snapshot` and `designs.graph`	n/a	Deliberately not a table (see 4.6)

4.6 A deliberate denormalisation: the graph and the result are documents

> The components and connections of a design, and the detailed result of a run, are stored as **jsonb documents** (`designs.graph`, `design_versions.snapshot`, `design_runs.payload`) instead of rows in a `run_component` table. Strictly, a document is not first normal form. This was chosen on purpose.

- A graph is edited, validated and saved as **one unit**. The strict schema (at most 50 components and 200 connections, 12 component types, identifiers checked) validates the whole document on every write, and the engine always reads it whole.
- A saved revision and a run must be **frozen**. If components lived in shared rows, a later change to the model or to the component list could silently alter an old result. A document per revision keeps each result tied to exactly the inputs that produced it (the fingerprint is a SHA-256 of that canonical document).

- Components are **never queried on their own** across designs, so no query needs a component table or its joins.
- The price is that PostgreSQL cannot enforce, say, that a connection points at an existing component. That rule is enforced by the schema and by the graph validation, and is tested.

Everything that is queried across rows (who owns what, which revision, which run belongs to which design, who may see it) is relational, keyed and constrained as described in Chapter 3.

5. List of APIs

Base URL: `http://localhost:5001/v1` (the Swagger UI is at `/docs`). All routes exchange JSON. The tables list all **115** endpoints: 49 GET, 53 POST, 8 PATCH and 5 DELETE. The list is generated from the controllers and was checked against the live OpenAPI document served by the running API: both contain exactly the same 115 operations.

Access is one of: **Public** (no token needed), **Signed in** (any valid access token; ownership is then checked inside each service, and another person's resource is answered as not found), or a **role** with the permission name the global `PermissionGuard` demands. The last column is the result of a real probe: every route was called once without a token and once with an ordinary user's token (a fresh, throw-away account on a test database), with placeholder ids and an empty body. `401 / 422` therefore reads: no token gives 401, and a signed-in ordinary user gets past the guards and is stopped by validation. `401 / 403` means the route needs a role permission. A public route answers something other than 401 without a token.

> **Probe summary.** 115 routes probed: 8 are public, 39 answered 403 to an ordinary user (they need a role permission), and the other 68 require sign-in only. Every one of the 39 permission routes in the code answered 403 and no other route did, so the declared permissions and the observed behaviour agree. `POST /v1/auth/refresh` is declared public (it needs no access token) but answers 401 without the refresh cookie.

5.1 Response format

Successful responses use one envelope; lists add `meta`. Errors are produced by one global exception filter. These bodies are real responses of the same requests shown in Figures 6.2, 6.5, 6.7, 6.8 and 6.9 (identifiers and times differ from run to run).

```
POST /v1/auth/register  ->  201 Created
{
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...",
    "expiresIn": 900,
    "user": { "id": "574bdcd8-654a-4047-bff4-5f4a661ad920",
              "email": "register-demo-...@example.test",
              "displayName": "Register Demo", "role": "user",
              "createdAt": "2026-11-09T07:37:08.760Z" }
  }
}
```

```
GET /v1/designs?page=1&pageSize=2  ->  200 OK
{ "data": [ { "id": "...", "name": "Shop MVP C", "revision": 1, "nodeCount": 3,
              "edgeCount": 2, "latestRun": null }, ... ],
  "meta": { "page": 1, "pageSize": 2, "total": 3 } }
```

```
{ "error": {
    "code": "VALIDATION_FAILED",
    "message": "Validation failed",
    "details": { "fields": [
      { "path": "name", "message": "Too small: expected string to have >=1 characters" },
      { "path": "",     "message": "Unrecognized key: \"unexpectedField\"" } ] } } }   // 422
```

```
{ "error": { "code": "CONFLICT",
    "message": "The saved design changed. Your draft has been retained.",
    "details": { "reason": "REVISION_MISMATCH", "currentRevision": 1 } } }   // 409

{ "error": { "code": "FORBIDDEN", "message": "You do not have permission to do that" } }   // 403
```

Status	Used when
200 / 201 / 202 / 204	Succeeded / created / accepted for asynchronous work (runs, submissions) / succeeded with no body (deletes, logout)
401	Token missing, invalid, expired (`SESSION_EXPIRED`) or revoked; wrong login credentials
403	Valid token but the role lacks the permission, or the account is suspended
404	Resource not found, or it belongs to someone else (the two look the same on purpose)
409	Stale revision, reused request id, an open report with the same title, a unique value that already exists, the last Admin
422	Validation failed: a field is wrong, too long or unknown; the graph is invalid; a body is too large or malformed
429	Rate limit, report limit, or a calculation already pending for the account
500 / 503	Unexpected error (generic message, details only in the server log) / a required service such as PostgreSQL, Redis or the calculation service is unavailable

5.2 Endpoints

Auth (6)

Method	Endpoint	Description	Access	No token / user
POST	`/v1/auth/register`	Register an account (role User)	Public	422 / 422
POST	`/v1/auth/login`	Log in; returns an access token and sets the refresh cookie	Public	422 / 422
POST	`/v1/auth/refresh`	Rotate the refresh token and issue a new access token	Public (refresh cookie)	401 / 401
POST	`/v1/auth/logout`	Log out and revoke the tokens	Public	204 / 204
POST	`/v1/auth/password`	Change the password; ends all earlier sessions	Signed in	401 / 422
GET	`/v1/auth/me`	Profile of the signed-in user	Signed in	401 / 200

Account (4)

Method	Endpoint	Description	Access	No token / user
PATCH	`/v1/account/profile`	Change own display name	Signed in	401 / 422
GET	`/v1/account/export`	Export everything the account owns as JSON	Signed in	401 / 200
POST	`/v1/account/delete`	Delete own account after a typed confirmation (204)	Signed in	401 / 422
GET	`/v1/account/roles`	Roles of the signed-in account	Signed in	401 / 200

Starter data and library (5)

Method	Endpoint	Description	Access	No token / user
GET	`/v1/templates`	The six starter templates	Signed in	401 / 200
GET	`/v1/simulation/catalog`	The 12 component types and their defaults	Signed in	401 / 200
GET	`/v1/dashboard`	The signed-in user's dashboard data	Signed in	401 / 200
GET	`/v1/library/designs`	List own designs: search, filters, sorting, paging	Signed in	401 / 200
GET	`/v1/health`	Liveness: PostgreSQL and Redis answer	Public	200 / 200

Designs, versions, notes and runs (26)

Method	Endpoint	Description	Access	No token / user
POST	`/v1/designs/prepare`	Build a draft from a text description or an import without saving it	Signed in	401 / 422
POST	`/v1/designs`	Create a design (blank, template, import or description)	Signed in	401 / 422
GET	`/v1/designs`	List own designs (page, pageSize)	Signed in	401 / 200
GET	`/v1/designs/:id`	Read one own design	Signed in	401 / 404
GET	`/v1/designs/:id/versions`	List the immutable versions	Signed in	401 / 404
GET	`/v1/designs/:id/report`	Report data: design, one revision and its latest run	Signed in	401 / 404
GET	`/v1/designs/:id/compare`	Compare two revisions	Signed in	401 / 404
GET	`/v1/designs/:id/runs`	List runs of a design	Signed in	401 / 404
POST	`/v1/designs/:id/versions/:revision/restore`	Restore an old version as a new version	Signed in	401 / 422
POST	`/v1/designs/:id/duplicate`	Duplicate a design	Signed in	401 / 404
POST	`/v1/designs/:id/archive`	Archive a design (reversible)	Signed in	401 / 404
POST	`/v1/designs/:id/unarchive`	Restore an archived design	Signed in	401 / 404
GET	`/v1/designs/:id/notes`	List notes	Signed in	401 / 404
POST	`/v1/designs/:id/notes`	Add a decision or assumption note	Signed in	401 / 422
PATCH	`/v1/designs/:id/notes/:noteId`	Edit a note	Signed in	401 / 422
DELETE	`/v1/designs/:id/notes/:noteId`	Delete a note (204)	Signed in	401 / 404
PATCH	`/v1/designs/:id`	Save a draft; requires expectedRevision (409 if stale)	Signed in	401 / 422
DELETE	`/v1/designs/:id`	Delete a design and everything under it (204)	Signed in	401 / 404
POST	`/v1/designs/:id/validate`	Validate the graph and list problems	Signed in	401 / 422
POST	`/v1/designs/:id/simulate`	Run a stress test (202 Accepted, asynchronous)	Signed in	401 / 422
POST	`/v1/designs/:id/fixes`	Suggest up to three server-verified fixes	Signed in	401 / 422
POST	`/v1/designs/:id/ladder`	Run the load ladder at 2x, 5x and 10x	Signed in	401 / 422
POST	`/v1/designs/:id/what-if`	Ask a what-if question on a copy of the draft	Signed in	401 / 422
POST	`/v1/designs/:id/limits`	Search for the breaking load (202 Accepted)	Signed in	401 / 422
GET	`/v1/designs/:id/runs/:runId`	Read one run (poll until completed or failed)	Signed in	401 / 404
POST	`/v1/simulation/preview`	Live preview of an unsaved graph (own rate-limit bucket)	Signed in	401 / 422

Share links (4)

Method	Endpoint	Description	Access	No token / user
POST	`/v1/designs/:id/shares`	Create a read-only share link (token shown once)	Signed in	401 / 422
GET	`/v1/designs/:id/shares`	List share links	Signed in	401 / 404
POST	`/v1/designs/:id/shares/:shareId/revoke`	Revoke a share link	Signed in	401 / 404
POST	`/v1/shares/open`	Open a shared view with a token (no sign-in)	Public	404 / 404

Reviews (11)

Method	Endpoint	Description	Access	No token / user
POST	`/v1/designs/:id/reviews`	Grant a Reviewer access to one revision	Signed in	401 / 422
GET	`/v1/designs/:id/reviews`	List review grants of a design	Signed in	401 / 404
POST	`/v1/designs/:id/reviews/:grantId/revoke`	Revoke a review grant	Signed in	401 / 404
GET	`/v1/designs/:id/reviews/:grantId/comments`	Owner reads the comment thread	Signed in	401 / 404
POST	`/v1/designs/:id/reviews/:grantId/comments`	Owner replies in the thread	Signed in	401 / 422
GET	`/v1/reviews/shared-with-me`	Reviewer lists designs shared with them	Reviewer (`REVIEW_READ`)	401 / 403
GET	`/v1/reviews/:id`	Reviewer reads the shared revision	Reviewer (`REVIEW_READ`)	401 / 403
GET	`/v1/reviews/:id/comments`	Reviewer reads the thread	Reviewer (`REVIEW_READ`)	401 / 403
POST	`/v1/reviews/:id/comments`	Reviewer adds a comment	Reviewer (`REVIEW_READ`)	401 / 403
PATCH	`/v1/reviews/:id/comments/:commentId`	Reviewer edits own comment	Reviewer (`REVIEW_READ`)	401 / 403
DELETE	`/v1/reviews/:id/comments/:commentId`	Reviewer removes own comment	Reviewer (`REVIEW_READ`)	401 / 403

Challenges (16)

Method	Endpoint	Description	Access	No token / user
POST	`/v1/instructor/challenges`	Create a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
GET	`/v1/instructor/challenges`	List own challenges	Instructor (`CHALLENGE_MANAGE`)	401 / 403
GET	`/v1/instructor/challenges/:id`	Read one own challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
PATCH	`/v1/instructor/challenges/:id`	Edit a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
POST	`/v1/instructor/challenges/:id/publish`	Publish a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
POST	`/v1/instructor/challenges/:id/archive`	Archive a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
POST	`/v1/instructor/challenges/:id/duplicate`	Duplicate a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
GET	`/v1/instructor/challenges/:id/submissions`	List submissions to a challenge	Instructor (`CHALLENGE_MANAGE`)	401 / 403
GET	`/v1/instructor/submissions/:id`	Read one submission	Instructor (`CHALLENGE_MANAGE`)	401 / 403
POST	`/v1/instructor/submissions/:id/feedback`	Give feedback on a submission	Instructor (`CHALLENGE_MANAGE`)	401 / 403
GET	`/v1/challenges`	List published challenges (public)	Public	200 / 200
GET	`/v1/challenges/:id`	Read one published challenge (public)	Public	404 / 404
POST	`/v1/challenges/:id/attempt`	Start an attempt at a challenge	Signed in	401 / 404
GET	`/v1/attempts/:id`	Read own attempt	Signed in	401 / 404
POST	`/v1/attempts/:id/submissions`	Submit an attempt (202 Accepted)	Signed in	401 / 422

Method	Endpoint	Description	Access	No token / user
GET	/v1/progress/challenges	Own progress across challenges	Signed in	401 / 200

Support (17)

Method	Endpoint	Description	Access	No token / user
GET	/v1/support/saved-replies	Staff: list saved replies (personal and team)	Support (SUPPORT_INBOX_REPLY)	401 / 403
POST	/v1/support/saved-replies	Staff: create a saved reply	Support (SUPPORT_INBOX_REPLY)	401 / 403
PATCH	/v1/support/saved-replies/:id	Staff: edit a saved reply (expectedUpdatedAt)	Support (SUPPORT_INBOX_REPLY)	401 / 403
DELETE	/v1/support/saved-replies/:id	Staff: delete a saved reply	Support (SUPPORT_INBOX_REPLY)	401 / 403
POST	/v1/support/saved-replies/:id/preview	Staff: fill a saved reply with a report's details (not sent)	Support (SUPPORT_INBOX_REPLY)	401 / 403
POST	/v1/support/reports	Send a problem report (201)	Signed in	401 / 422
GET	/v1/support/reports	List own reports	Signed in	401 / 200
GET	/v1/support/reports/:id	Read own report and its public thread	Signed in	401 / 404
POST	/v1/support/reports/:id/messages	Reply to own report	Signed in	401 / 422
POST	/v1/support/inbox/:id/merge	Staff: merge a duplicate report into another	Support (SUPPORT_INBOX_REPLY)	401 / 403
POST	/v1/support/inbox/:id/unmerge	Staff: undo a merge	Support (SUPPORT_INBOX_REPLY)	401 / 403
GET	/v1/support/agents	Staff: list assignable Support accounts	Support (SUPPORT_INBOX_READ)	401 / 403
GET	/v1/support/inbox	Staff: list and search all reports (cursor paging)	Support (SUPPORT_INBOX_READ)	401 / 403
GET	/v1/support/inbox/:id	Staff: read a report with internal notes	Support (SUPPORT_INBOX_READ)	401 / 403
POST	/v1/support/inbox/:id/read	Staff: mark a report as read	Support (SUPPORT_INBOX_READ)	401 / 403
POST	/v1/support/inbox/:id/messages	Staff: reply or add an internal note	Support (SUPPORT_INBOX_REPLY)	401 / 403
PATCH	/v1/support/inbox/:id	Staff: set status, priority or assignee	Support (SUPPORT_INBOX_REPLY)	401 / 403

Notifications (7)

Method	Endpoint	Description	Access	No token / user
GET	/v1/notifications	List own notifications (cursor paging)	Signed in	401 / 200
GET	/v1/notifications/unread-count	Number of unread notifications	Signed in	401 / 200
GET	/v1/notifications/preferences	Own per-kind mute settings	Signed in	401 / 200
PATCH	/v1/notifications/preferences	Mute or unmute a kind	Signed in	401 / 422
POST	/v1/notifications/read-all	Mark all as read	Signed in	401 / 200
POST	/v1/notifications/:id/read	Mark one as read	Signed in	401 / 404
POST	/v1/notifications/:id/open	Mark as read and return the target page	Signed in	401 / 404

Admin (10)

Method	Endpoint	Description	Access	No token / user
GET	`/v1/admin/metrics`	Admin: dashboard snapshot (aggregate counts only)	Admin (`ADMIN_METRICS_READ`)	401 / 403
GET	`/v1/admin/metrics/export`	Admin: export a snapshot as CSV or JSON	Admin (`ADMIN_METRICS_READ`)	401 / 403
GET	`/v1/admin/users`	Admin: search accounts	Admin (`ADMIN_USERS_READ`)	401 / 403
POST	`/v1/admin/users/:id/suspend`	Admin: suspend an account	Admin (`ADMIN_USERS_SUSPEND`)	401 / 403
POST	`/v1/admin/users/:id/restore`	Admin: restore an account	Admin (`ADMIN_USERS_SUSPEND`)	401 / 403
GET	`/v1/admin/users/lookup`	Admin: look up an account by email	Admin (`ADMIN_USERS_READ`)	401 / 403
GET	`/v1/admin/users/:id/roles`	Admin: roles of an account	Admin (`ADMIN_USERS_READ`)	401 / 403
POST	`/v1/admin/users/:id/roles`	Admin: grant a role	Admin (`ADMIN_ROLES_GRANT`)	401 / 403
DELETE	`/v1/admin/users/:id/roles/:role`	Admin: revoke a role	Admin (`ADMIN_ROLES_GRANT`)	401 / 403
GET	`/v1/admin/audit`	Admin: read the append-only audit log (cursor paging)	Admin (`ADMIN_AUDIT_READ`)	401 / 403

AI (optional) (9)

Method	Endpoint	Description	Access	No token / user
GET	`/v1/ai/tasks/readiness`	Which AI tasks are switched on	Signed in	401 / 200
POST	`/v1/ai/tasks/describe`	Turn a description into a draft graph (AI or keywords)	Signed in	401 / 422
POST	`/v1/ai/tasks/fix-intent`	Turn a goal sentence into a filter for fixes	Signed in	401 / 422
POST	`/v1/ai/tasks/what-if`	Turn a sentence into a what-if change set	Signed in	401 / 422
GET	`/v1/ai/readiness`	Whether the AI provider is available	Signed in	401 / 200
POST	`/v1/ai/jobs`	Submit an AI job for a design	Signed in	401 / 422
GET	`/v1/ai/jobs/:id`	Read an AI job	Signed in	401 / 404
GET	`/v1/ai/jobs/:id/evidence`	Read the facts an AI job used	Signed in	401 / 404
POST	`/v1/ai/jobs/:id/cancel`	Cancel an AI job	Signed in	401 / 404

5.3 Requirement coverage

Where each requirement of the course guide is implemented, and where STRESSFORGE differs from the guide's suggested tools.

Requirement	Implementation	Note
Minimum requirements		
Multiple entities with CRUD	Designs (create, read list and by id, update, delete), notes, review comments and saved replies have full CRUD; challenges, reports, shares and grants have create, read and state-changing endpoints; audit logs are read-only. Lists are paged.	24 application tables; challenges are archived, not deleted
Database integration	TypeORM and PostgreSQL; 34 foreign keys; migrations only (`synchronize` is off)	

Requirement	Implementation	Note
Data transaction	Creating a report writes ticket, activity and notifications in one transaction; saving, running, merging and role changes use transactions, advisory locks and row locks	Figures 6.10, 6.10b and 6.11
JWT authentication and authorization	`AuthGuard`, rotating refresh tokens, `RolesGuard`, and a database-backed `PermissionGuard` with five roles	Figures 6.3 to 6.5
Input validation	Strict Zod schemas through a global validation pipe; unknown fields are rejected; failures name each field	**Differs from the guide:** Zod instead of class-validator, and **422** instead of 400 (Figure 6.7)
HTTP exception handling	One global filter and one error shape; stacks are never returned; a unique-key violation that reaches the filter becomes 409	Figure 6.8 shows a 409
Bonus requirements		
Middleware	`RequestLoggerMiddleware` (logs method, path, status and duration only, never query strings, headers or bodies)	
Custom decorators	`@Public()`, `@CurrentUser()`, `@RequirePermission()`, `@Roles()`, `@PreviewThrottle()`	
Mailer	Optional email through nodemailer, a Bull repeating job and the `notification_emails` outcome table; off by default	Figure 6.14; Mailtrap sandbox
Swagger UI	Served at `/docs` with bearer authentication; 115 operations	Chapter 6
Interceptors	`ResponseEnvelopeInterceptor` wraps every success in `{ data }`	One interceptor only
Audit log	`role_audit` (every role change and suspension) and `support_activity` (every report change), both append-only by trigger	**Partial:** not every create, update and delete is audited
Env	Zod-validated configuration at start-up; `.env.example` is committed; `.env` is ignored by Git	
Migrations	21 TypeORM migrations; a fresh database is built by running them (Figure 6.15)	
Response manipulation	Envelope with paging metadata; the password hash is never selected into a response	
Database transaction (advanced)	Advisory locks per account, row locks, REPEATABLE READ read-only snapshots for reports and exports, optimistic concurrency, idempotency keys	

> **Deviations from the guide's suggested tools, stated openly.** The guide assumes bcrypt and class-validator; this project uses **argon2** (a stronger password hash) and **Zod** (validation and types from one schema). The guide says validation errors return 400; this API returns **422**. The audit bonus is partial, as stated above. The guide allows equivalent tools if the instructor accepts them. **Confirm the instructor's acceptance in writing and note it here.**

5.4 Roles and permissions

One table (`common/enums/account-role.enum.ts`) maps each role to the permissions it holds. There is no inheritance. Roles are read from the `user_roles` table on every guarded call, never from the token.

Permission	Allows	Held by
`admin.users.read`	Search and look up accounts, read an account's roles	Admin
`admin.roles.grant`	Grant and revoke roles	Admin
`admin.users.suspend`	Suspend and restore accounts	Admin
`admin.audit.read`	Read the audit log	Admin
`admin.metrics.read`	Dashboard snapshot and exports (aggregate counts only)	Admin
`review.read`	Read designs shared with the account and comment	Reviewer

Permission	Allows	Held by
`challenge.manage`	Author, publish and archive challenges; read submissions; give feedback	Instructor
`support.inbox.read`	See every report and its thread	Support
`support.inbox.reply`	Reply, internal notes, status, priority, assignment, saved replies, merge	Support

6. Screenshots of Functionality

Each figure shows one function; the request URL, the method, the request body and the response status and body are visible. Failure cases are included (401, 403, 422, 409 and a rollback).

> **How these were captured.** The Swagger figures are screenshots of the real Swagger UI of the running API, produced by an automated Chrome session that clicked Try it out and Execute against a **throw-away copy of the API on port 5055** with a disposable test database (so the Request URL shows port 5055, not 5001). The accounts and tokens in them are test values. Figures 6.10b, 6.11 and 6.15 render real command output captured at the time shown in each image. Figure 6.14 is a normal screenshot of the Mailtrap web page. Each Swagger figure is cropped from the top of the operation to the end of the response body; the long response-header dump below it is left out.

Swagger UI served by the running API.

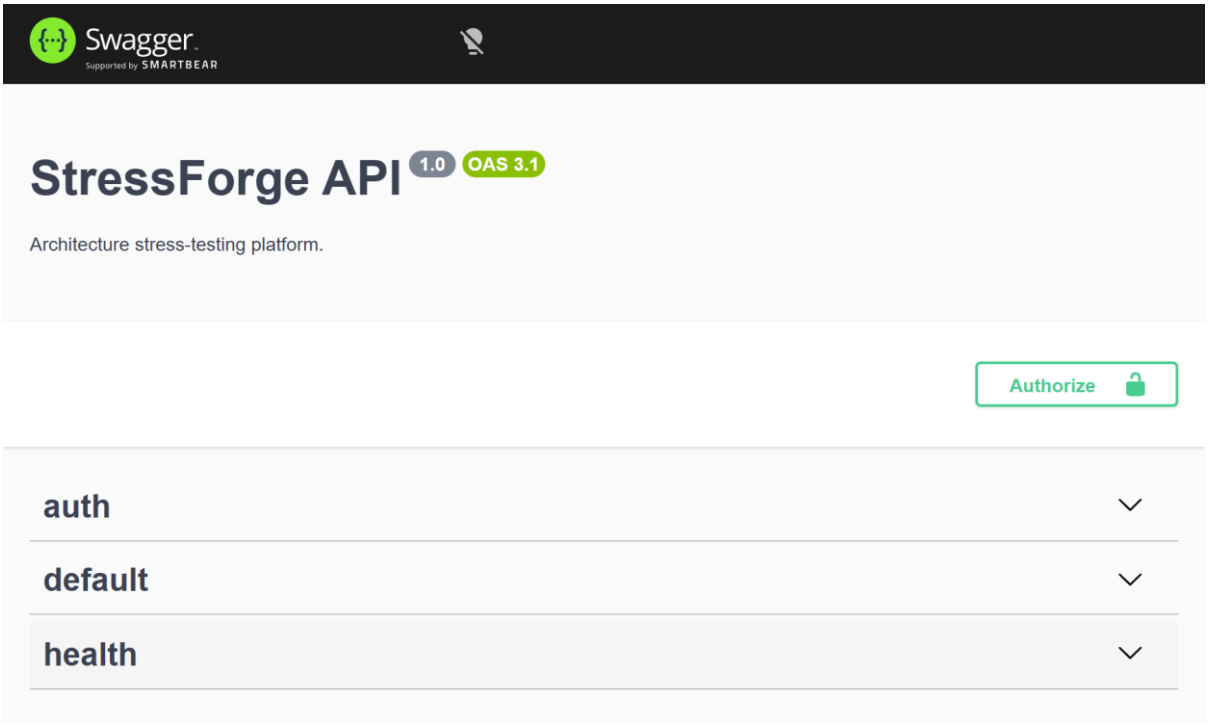


Figure 6.1: Swagger UI at /docs: the endpoint groups and the Authorize button

Registration returns a token and the user, never the hash.

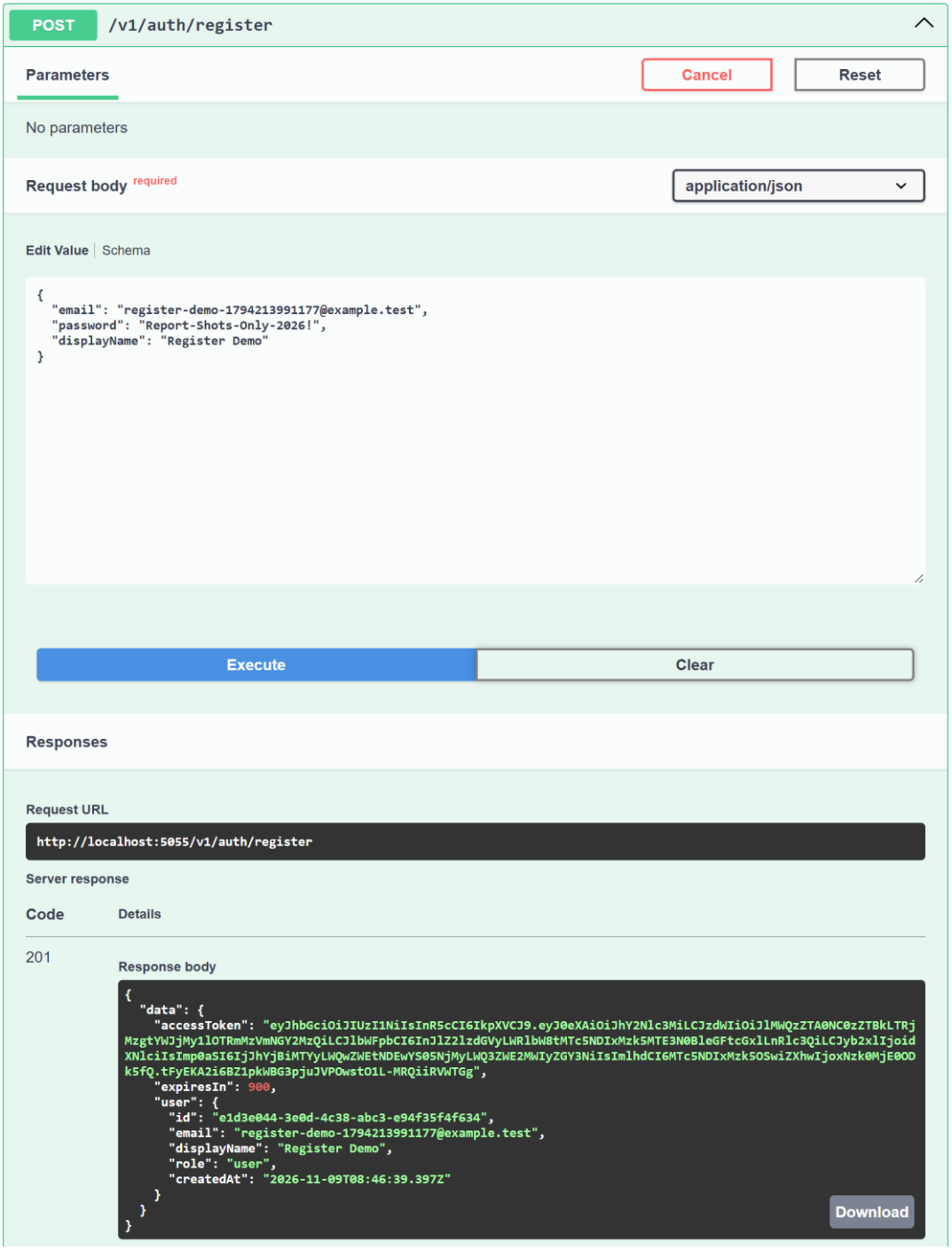


Figure 6.2: POST /v1/auth/register, 201 Created; the response contains no password hash

Login with the account created in the same session.

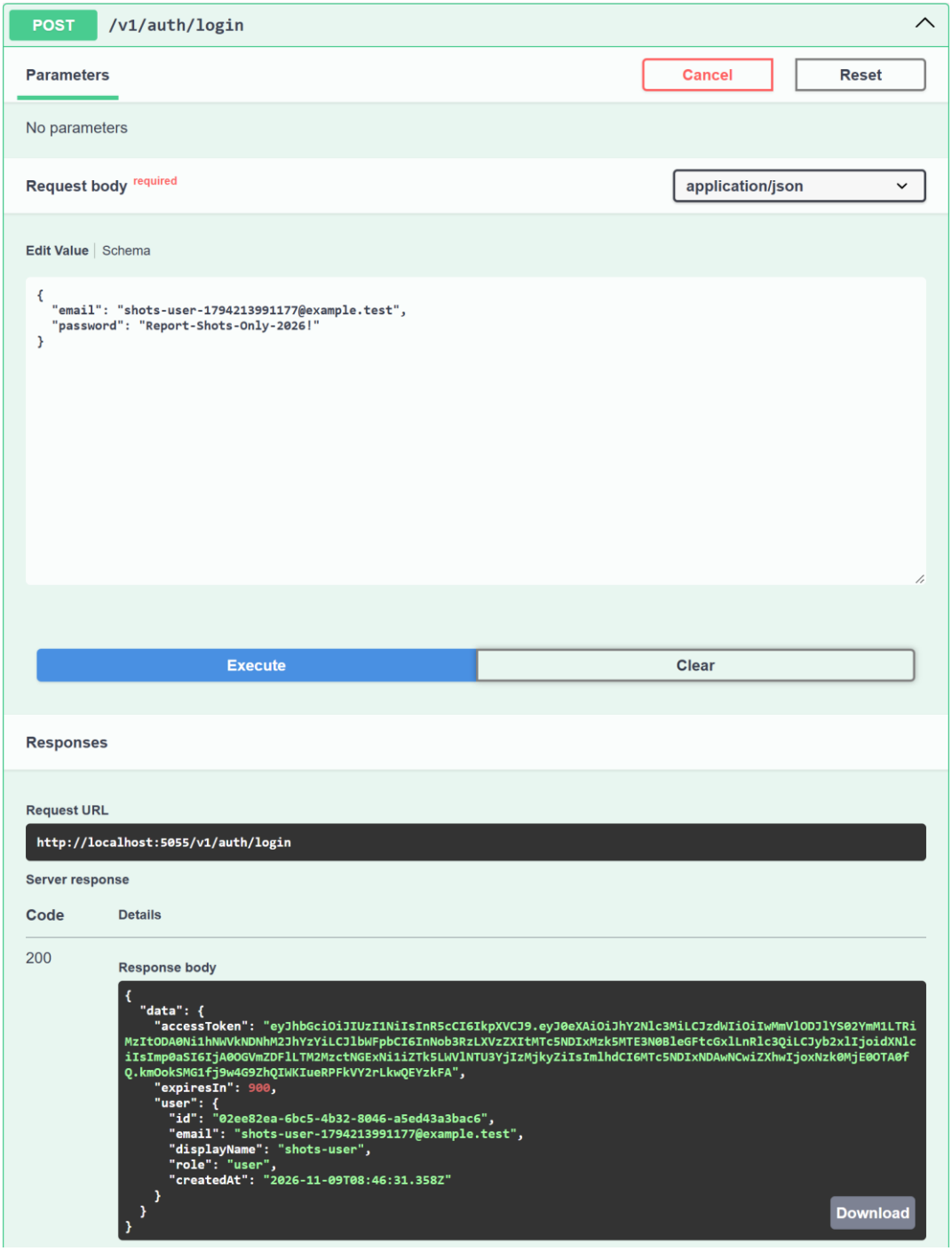


Figure 6.3: POST /v1/auth/login returns a JWT access token

A protected route refuses a request without a token.

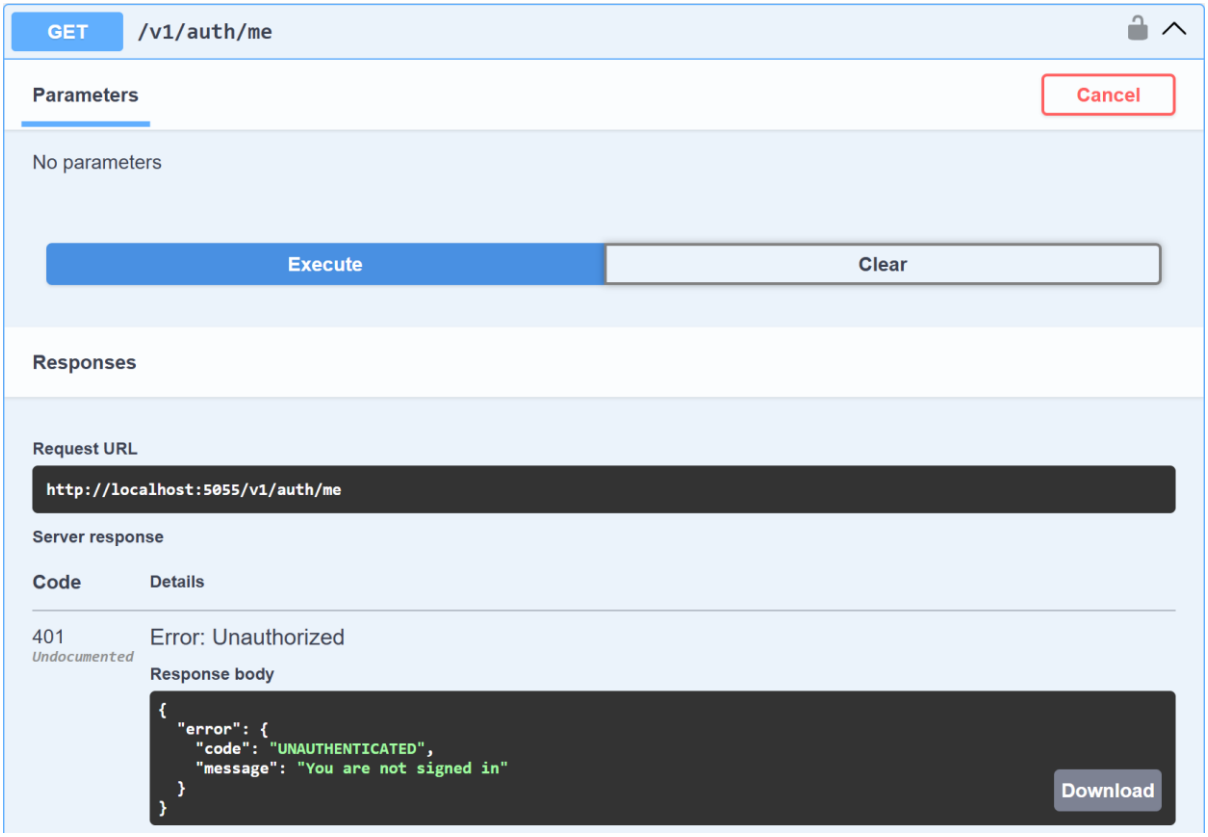


Figure 6.4: GET /v1/auth/me without a token, 401 Unauthorized

A valid token without the Admin permission is refused.

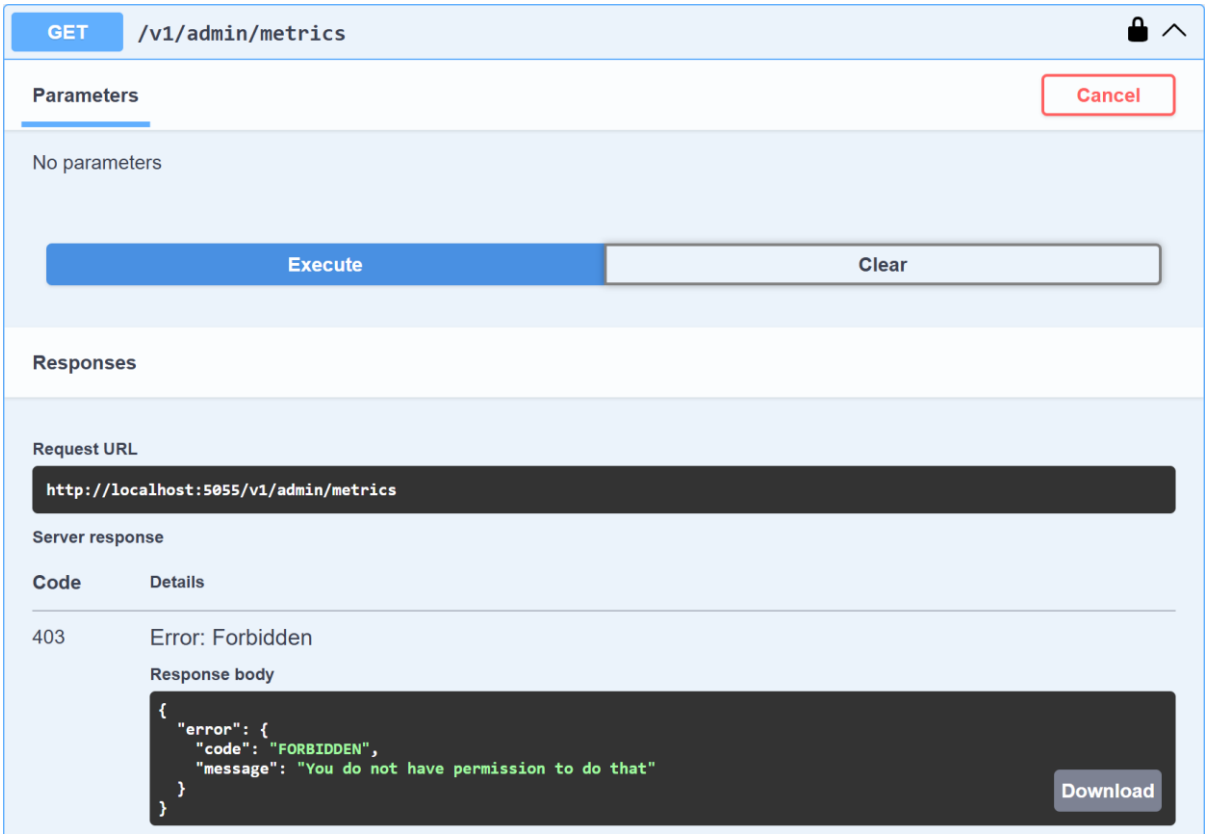


Figure 6.5: an ordinary user calls GET /v1/admin/metrics, 403 Forbidden

Creating a design (the guide's "create book").

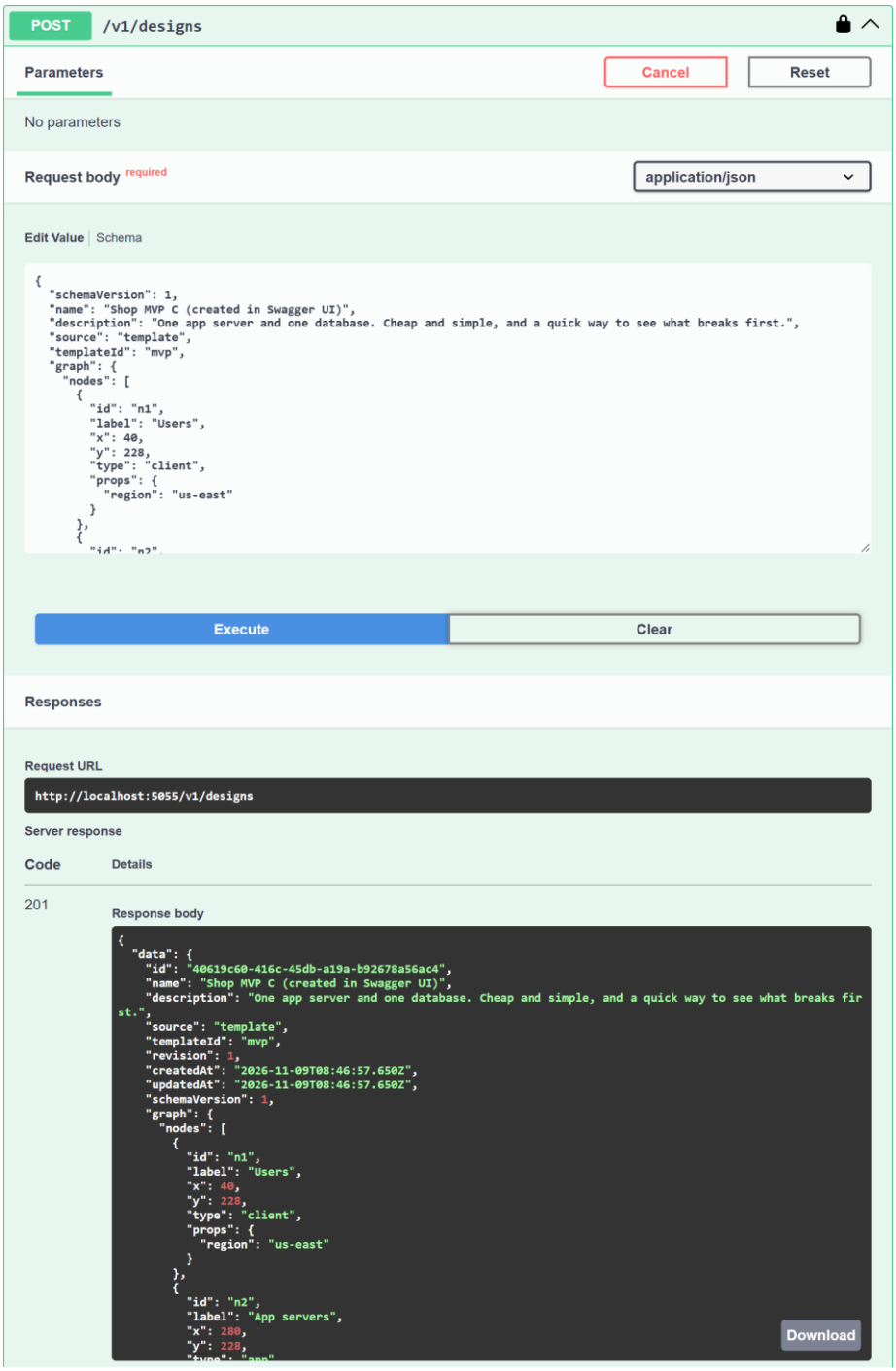


Figure 6.6: POST /v1/designs with the MVP template graph, 201 Created

Validation error. This API returns 422 where the guide says 400.

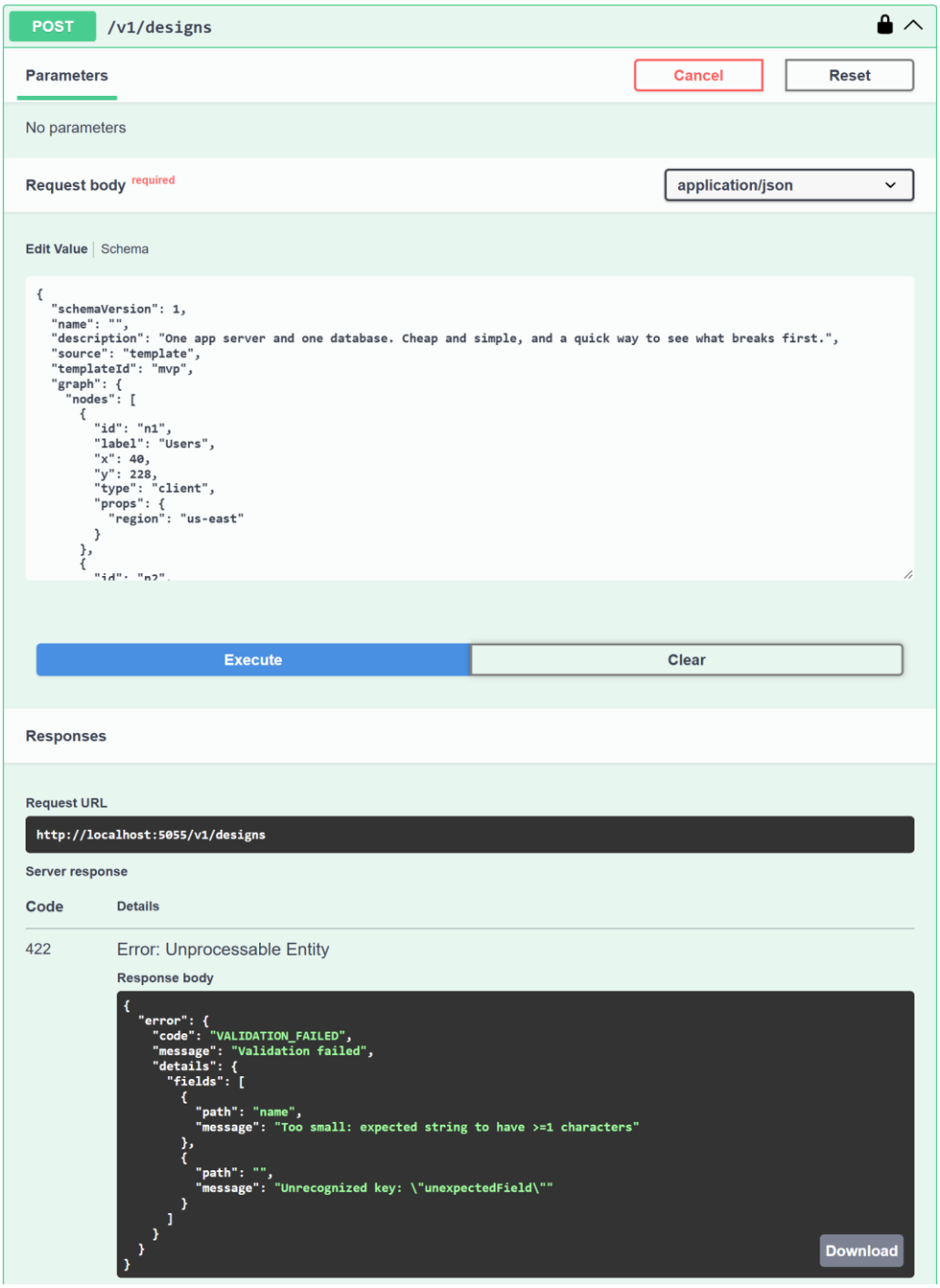


Figure 6.7: POST /v1/designs with an empty name and an unknown field, 422 with each failing field named

Optimistic-concurrency conflict (the guide's duplicate-value 409).

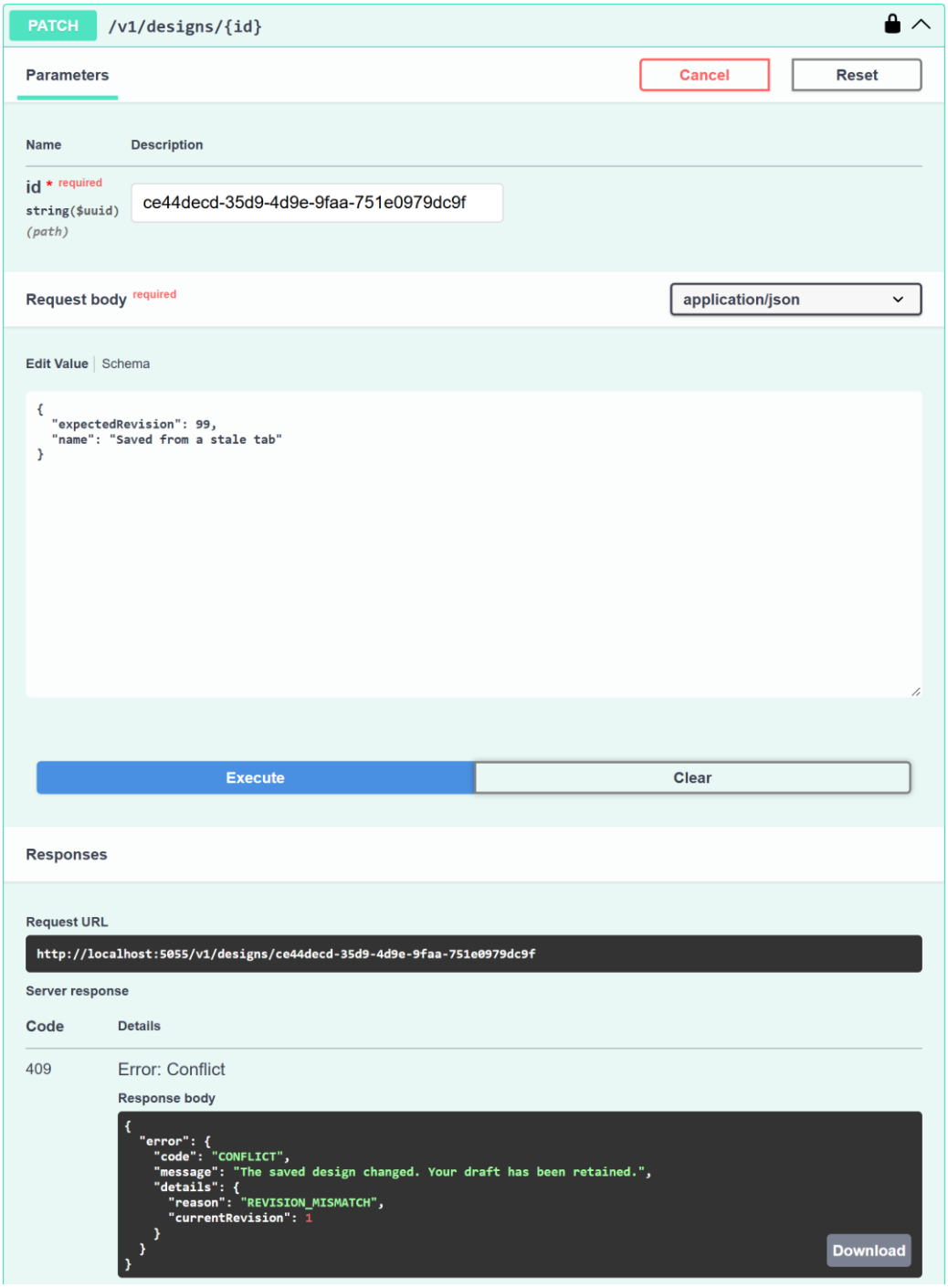


Figure 6.8: PATCH /v1/designs/{id} with a stale expectedRevision, 409 Conflict

List with pagination.

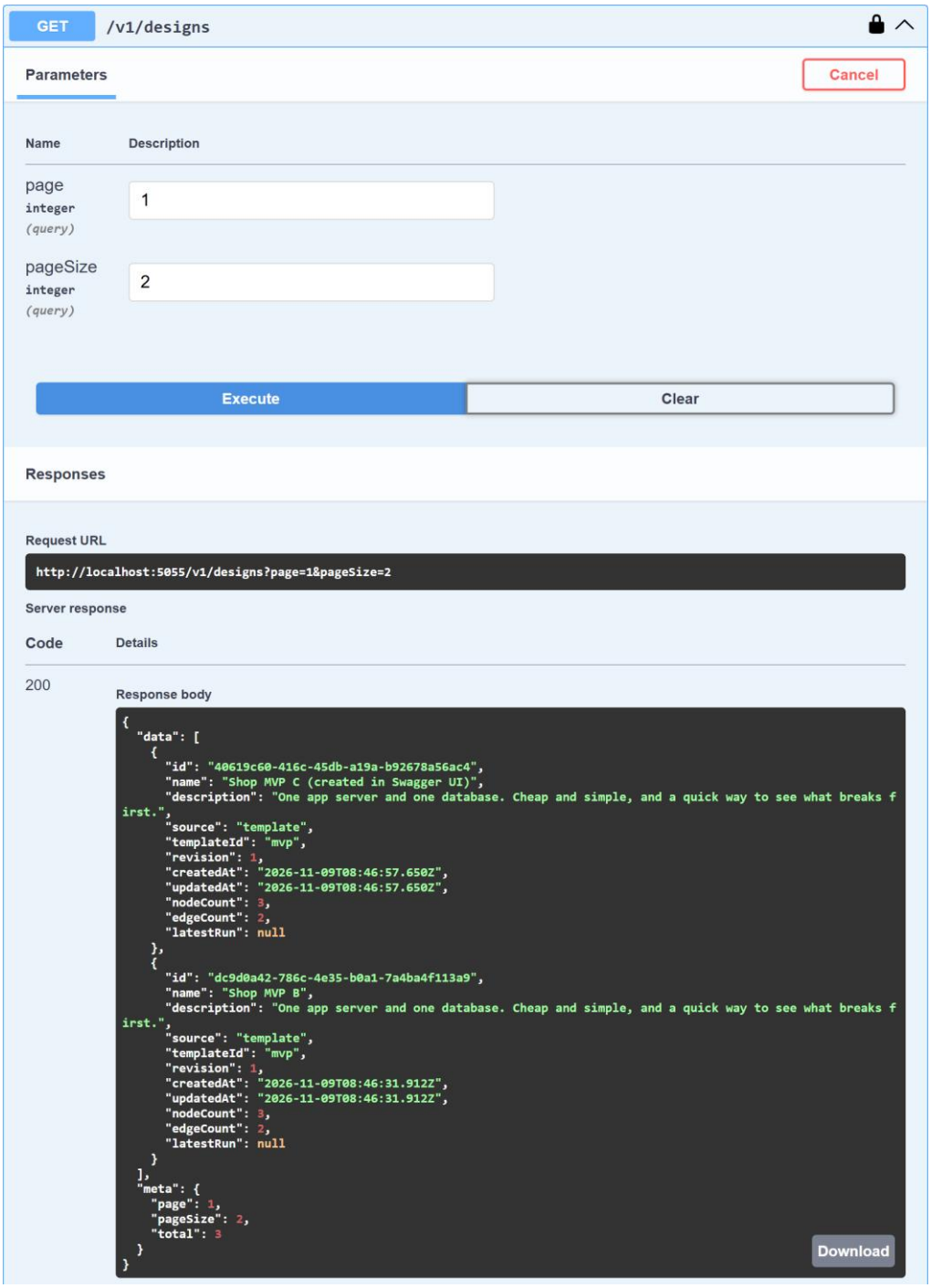


Figure 6.9: GET /v1/designs?page=1&pageSize=2 with the paging metadata

Transaction success, request side.

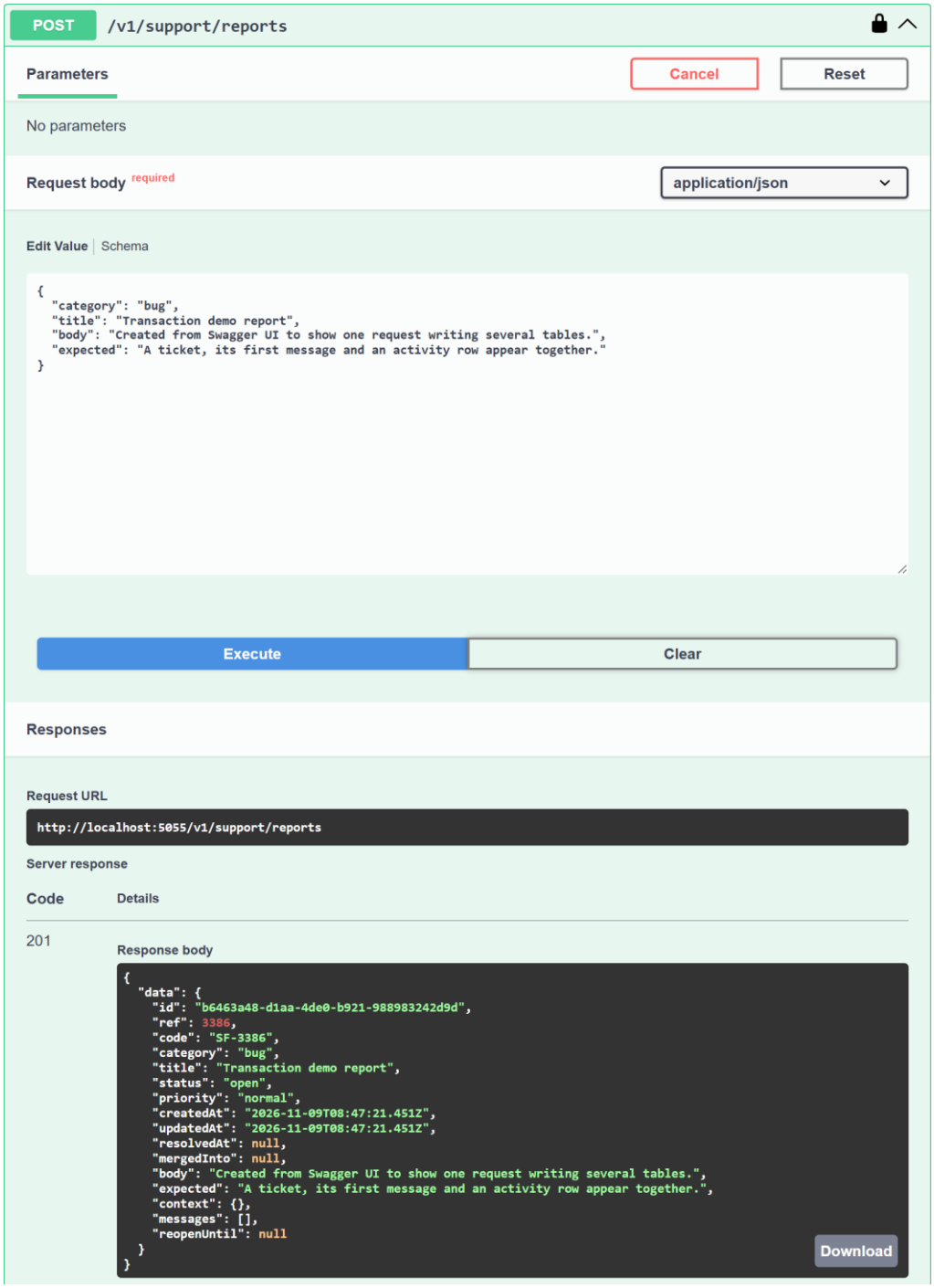


Figure 6.10: POST /v1/support/reports, 201 Created: one request, one transaction

Transaction success, database side. Real psql output, rendered from the terminal.

```
Real psql output, captured Mon Nov 09 2026 14:47:29. One POST /v1/support/reports wrote the ticket, its activity row
and the Support notifications in a single transaction.

PS C:\...\nest-boilerplate> psql -d stressforge_test -c "SELECT ref, category, status, priority, left(title,28) AS t
itle, created_at::timestamp(0) FROM support_tickets WHERE id='b6463a48-d1aa-4de0-b921-988983242d9d'"
 ref  | category | status | priority |          title          |     created_at
------+----------+--------+----------+-------------------------+---------------------
 3386 | bug      | open   | normal   | Transaction demo report | 2026-11-09 08:47:21
(1 row)


PS C:\...\nest-boilerplate> psql -d stressforge_test -c "SELECT action, actor_kind, at::timestamp(0) FROM support_ac
tivity WHERE ticket_id='b6463a48-d1aa-4de0-b921-988983242d9d' ORDER BY seq"
 action | actor_kind |         at
--------+------------+---------------------
 create | user       | 2026-11-09 08:47:21
(1 row)


PS C:\...\nest-boilerplate> psql -d stressforge_test -c "SELECT kind, count(*) AS rows_written FROM notifications WH
ERE ref_id='b6463a48-d1aa-4de0-b921-988983242d9d' GROUP BY kind"
        kind        | rows_written
--------------------+--------------
 support_new_report |           34
(1 row)
```

Figure 6.10b: the rows the same request wrote (ticket, activity row and Support notifications), read with psql

Rollback on failure. The failure is injected inside the server, so this is a test run, not a Swagger call.

```
Real test code and output, captured Mon Nov 09 2026 14:47:29. The test makes the notification insert fail in the mid
dle of the merge transaction, then checks that the merge, the system message and the audit row were all rolled back.

PS C:\...\nest-boilerplate> Get-Content test/support-extras.e2e-spec.ts | Select-Object -Skip 86 -First 5
 it('rolls back merge, system message and audit when the transactional notification insert fails',async()=>{
  const x=await report(a),y=await report(b);
  const fault=jest.spyOn(app.get(NotificationsService),'emit').mockRejectedValueOnce(new Error('deliberate event fai
lure'));

  try{expect((await merge(x.id,y.id)).status).toBe(500);
  }finally{fault.mockRestore();
  }
  expect((await detail(x.id)).mergedInto).toBeNull();
  expect((await db.query('SELECT count(*)::int AS n FROM support_messages WHERE ticket_id=$1',[x.id]))[0].n).toBe
(0);
  expect((await db.query("SELECT count(*)::int AS n FROM support_activity WHERE ticket_id=$1 AND action='merge'",[x.
id]))[0].n).toBe(0);

 });


PS C:\...\nest-boilerplate> pnpm test:e2e -t "rolls back merge, system message and audit when the transactional noti
fication insert fails"
Test Suites: 1 skipped, 1 passed, 1 of 2 total
Tests:       21 skipped, 1 passed, 22 total
```

Figure 6.11: a test forces a failure in the middle of the merge transaction and checks that nothing was saved

An asynchronous run accepted (stands in for the guide's "return book with late fine").

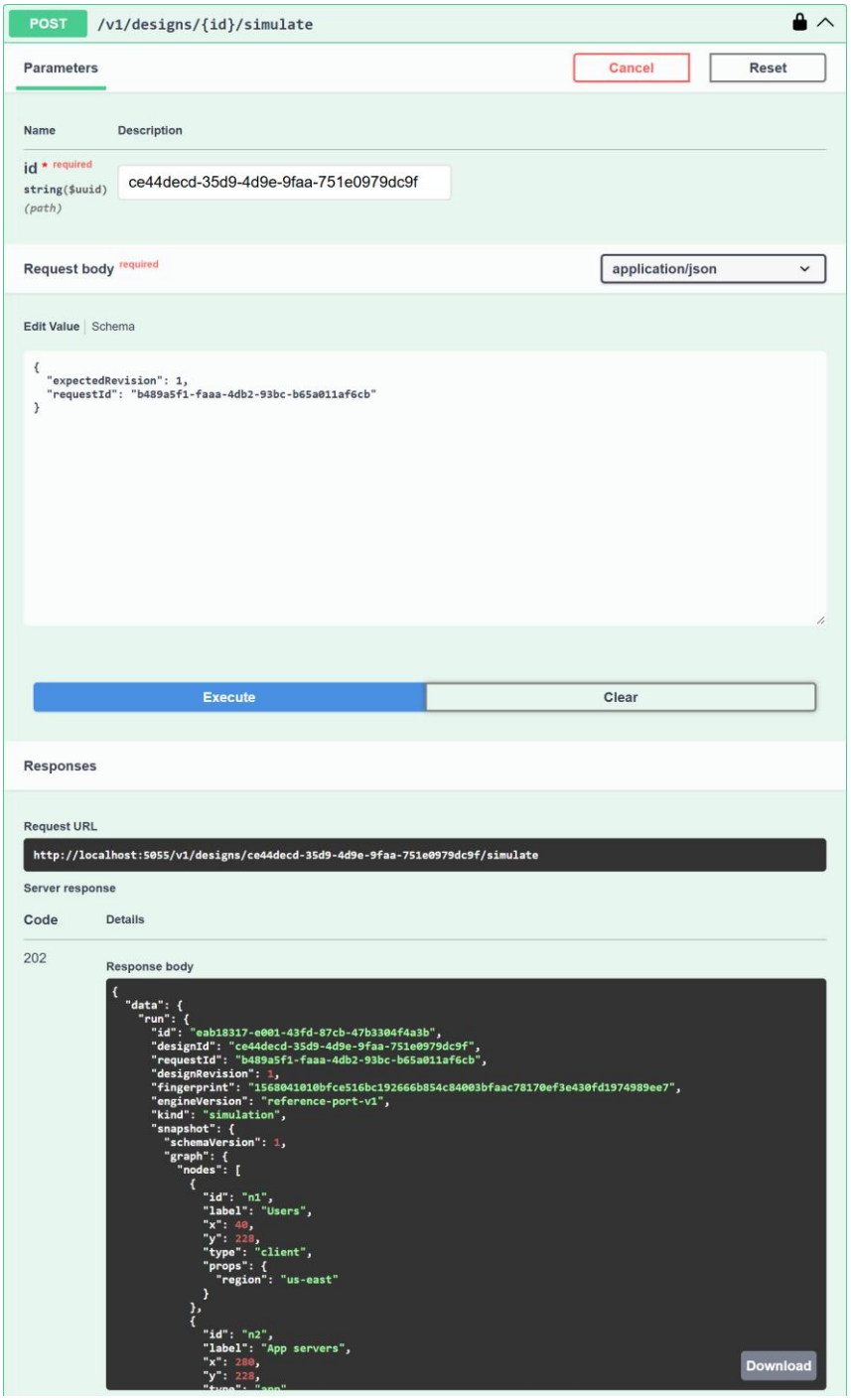


Figure 6.12: POST /v1/designs/{id}/simulate, 202 Accepted

Audit log listing.

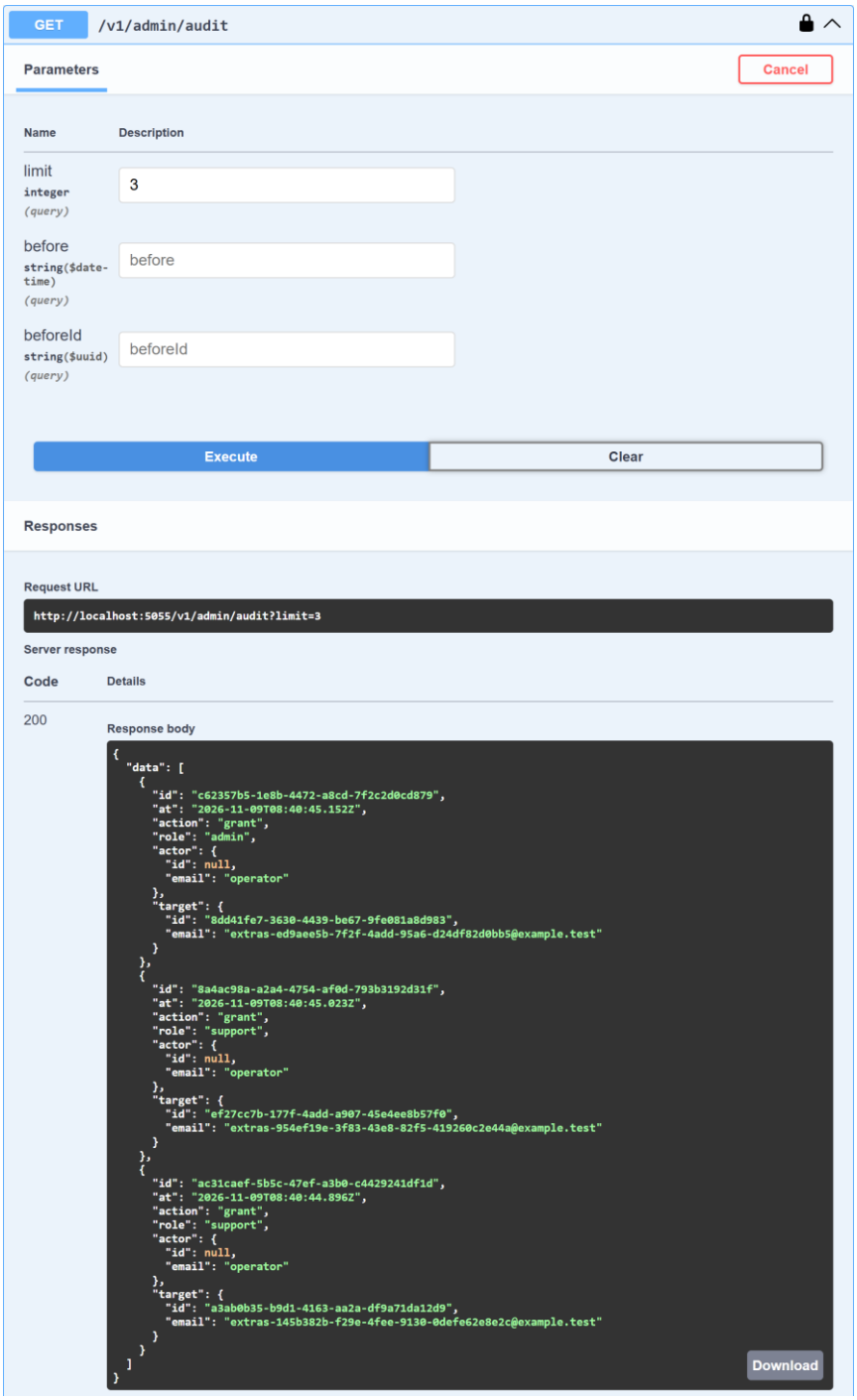


Figure 6.13: GET /v1/admin/audit?limit=3 as an Admin

Email in the Mailtrap inbox. The mail contains the report number and a link, never the text of the report.

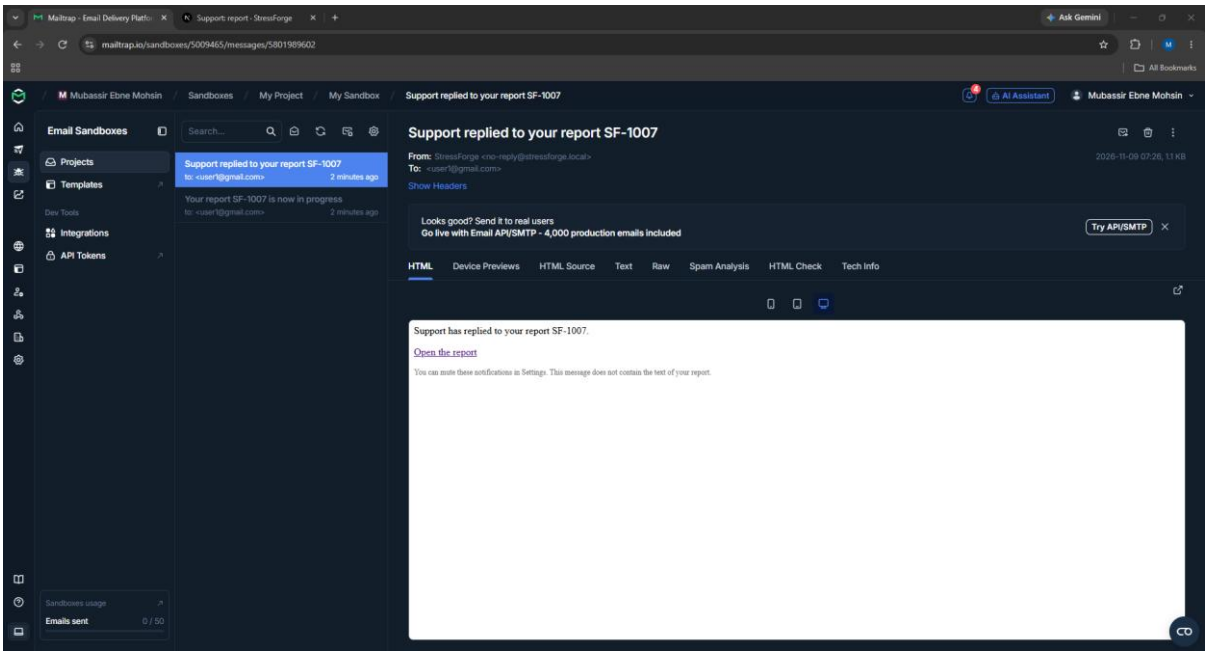


Figure 6.14: the email "Support replied to your report" in the Mailtrap sandbox inbox

Migrations on an empty database. Real terminal output, rendered; the database was deleted afterwards.

```
Real terminal output, captured Mon Nov 09 2026 14:47:29. The database was empty and was deleted again afterwards.

PS C:\...\nest-boilerplate> psql -c "CREATE DATABASE sf_shots_migrate"
CREATE DATABASE

PS C:\...\nest-boilerplate> pnpm migration:run   (DB_DATABASE=sf_shots_migrate)
Applied: CreateUsers1761000000000,
  CreateDesigns1792800000000,
  AiFoundation1793200000000,
  VersionRestore1793300000000,
  DesignNotes1793400000000,
  DesignArchive1793500000000,
  DesignShares1793600000000,
  AccountRoles1793700000000,
  AccountStatus1793800000000,
  ReviewGrants1793900000000,
  ReviewComments1794000000000,
  Challenges1794100000000,
  ChallengeAttempts1794200000000,
  ChallengeFeedback1794300000000,
  AccountSelfService1794400000000,
  AiDesignSource1794500000000,
  NoteComponent1794600000000,
  Support1794700000000,
  Notifications1794800000000,
  SupportExtras1794900000000,
  NotificationEmails1795000000000

PS C:\...\nest-boilerplate> psql -d sf_shots_migrate -c "SELECT count(*) AS tables FROM information_schema.tables
..."
 tables
--------
     25
(1 row)
```

Figure 6.15: pnpm migration:run on an empty database applies all 21 migrations

7. Implementation Highlights

The excerpts are copied from the repository; where lines are left out for length the omission is marked with `// ...`.

7.1 A strict request contract

A request body is described once, as a Zod schema. `z.strictObject` rejects any key that is not listed, so nothing extra (or sensitive) can be smuggled in; every text field has a minimum and maximum length and refuses control characters. The same schema supplies the TypeScript type and the Swagger documentation. File: `src/common/contracts/phase3/support.schema.ts`.

```
export const createReportSchema = z.strictObject({
  category: categorySchema,
  title: plain(TITLE_MIN, TITLE_MAX),
  body: plain(BODY_MIN, BODY_MAX),
  expected: plain(1, EXPECTED_MAX).optional(),
  context: reportContextSchema.default({}),
  /** Set after the person has been told they already have an open report with this title. */
  confirmDuplicate: z.boolean().optional(),
});
```

7.2 Creating a report inside a transaction

Creating a report counts the account's recent reports, checks for a duplicate title, inserts the ticket, writes the activity row and creates the notifications for Support staff. All of it runs in one transaction. A per-account **advisory lock** at the start makes two quick requests from the same account run one after the other, so both cannot slip under the limit of 5 reports an hour. If any step throws, PostgreSQL rolls everything back. File: `src/modules/support/support.service.ts`.

```
  async create(user: AuthUser, raw: unknown) {
    const parsed = createReportSchema.safeParse(raw);
    if (!parsed.success) throw invalid(parsed.error);
    const input = parsed.data;
    const id = await this.db.transaction(async (m) => {
      // One account's reports are counted and added one at a time, so two quick requests cannot both slip under
the limit.
      await m.query("SELECT pg_advisory_xact_lock(hashtext($1))", ["sf-support-create:" + user.id]);
      // ... counting query, then the two limit checks that throw AppException("RATE_LIMITED", ...)
      if (!input.confirmDuplicate) {
        // ... look for an open report with the same title; throw AppException("CONFLICT", ..., { reason:
"DUPLICATE_OPEN" })
      }
      const [row] = (await m.query(
        `INSERT INTO support_tickets (author_id, category, title, body, expected, context, created_at, updated_at,
last_user_activity_at)
         VALUES ($1, $2, $3, $4, $5, $6::jsonb, ${NOW}, ${NOW}, ${NOW}) RETURNING id, ref`,
        [user.id, input.category, input.title, input.body, input.expected ?? null, JSON.stringify(input.context)],
      )) as Row[];
      await this.activity(m, { id: row.id as string, ref: row.ref as number }, "create", { kind: "user", id:
user.id, email: user.email }, null, "open");
      await this.notifications.team(m,"support_new_report",row.id as string,user.id);
      return row.id as string;
    });
    return this.own(user.id, id);
  }
```

Figure 6.10b shows the rows this transaction wrote, and Figure 6.11 shows an end-to-end test that makes the notification step fail on purpose and then checks that the ticket changes, the system message and the audit row were all rolled back.

7.3 Permissions read from the database on every call

The global `PermissionGuard` reads the permission a route declares with `@RequirePermission` and compares it with the account's roles **loaded from the `user_roles` table at that moment**, not from the token. A role revoked a second ago therefore stops working immediately, even while the account still holds a valid access token. File: `src/modules/roles/permission.guard.ts`.

```
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const needed = this.reflector.getAllAndOverride<Permission | undefined>(PERMISSION_KEY,
[context.getHandler(), context.getClass()]);
    if (!needed) return true;
    const user = context.switchToHttp().getRequest<IRequest>().user;
    if (!user) throw new AppException("UNAUTHENTICATED", "You are not signed in");
    if (!hasPermission(await this.roles.rolesOf(user.id), needed)) throw new
AppException("FORBIDDEN", "You do not have permission to do that");
    return true;
  }
```

Guards are registered once in `app.module.ts` as global providers in a fixed order (rate limit, authentication, ranked role check, permission), so no controller can forget them.

7.4 Timing-safe login

A login for an email that does not exist still performs an argon2 verification against a dummy hash, so the response time does not reveal which emails have accounts. A suspended account is reported only after the password is correct, for the same reason. File: `src/modules/auth/auth.service.ts`.

```
const DUMMY_HASH_PROMISE = argon2.hash("not-a-real-password");
// ...
    const user = await this.users.findByEmail(dto.email);
    const hash = user?.passwordHash ?? (await DUMMY_HASH_PROMISE);
    const ok = await argon2.verify(hash, dto.password);
    if (!user || !ok)
      throw new AppException("UNAUTHENTICATED", BAD_CREDENTIALS);
```

7.5 One exception filter for every error

Every failure leaves the server through one global filter, so the client sees one shape and never a stack trace. A unique-key violation that reaches the filter (a race, or a path that did not check first) becomes **409 Conflict** with a generic message; constraint and column names stay in the database. File: `src/common/filters/all-exceptions.filter.ts`.

```
/** PostgreSQL error 23505 (unique_violation), whether it arrives wrapped by TypeORM or straight from
pg. */
export function isUniqueViolation(exception: unknown): boolean {
  if (!(exception instanceof Error)) return false;
  const direct = (exception as { code?: unknown }).code;
  const wrapped = (exception as { driverError?: { code?: unknown } })
    .driverError?.code;
  return direct === "23505" || wrapped === "23505";
}
// ...
    if (isUniqueViolation(exception)) {
      return this.build(409, "CONFLICT", "That value already exists");
    }
    return this.build(500, "INTERNAL", "Something went wrong");
```

7.6 Optional email that cannot hurt a request

The mailer is a Bull repeating job that **reads** notifications which already exist, sends a short email for the two reporter-facing kinds, and records the outcome in `notification_emails`. It edits no existing write path and

never runs inside a request, so a mail failure can delay an email but never affect a Support action. The module is not even loaded unless `MAIL_ENABLED=true`. File: `src/modules/mails/mail-outbox.service.ts`.

```
      try {
        const message = renderMail(row.kind, {
          ref: row.ref,
          status: row.status,
          url: reportUrl(this.options.frontendUrl, row.ref_id),
        });
        await this.sender.send({ to: row.email, ...message });
        await this.record(row.id, "sent", attempts, null);
        result.sent++;
      } catch (error) {
        const code = String((error as { code?: unknown })?.code ?? "SEND_FAILED").slice(0, 40);
        const final = attempts >= MAX_ATTEMPTS;
        await this.record(row.id, final ? "failed" : "retry", attempts, code);
        // No address, subject or server message: those can carry personal data or credentials.
        this.log.warn(`Email not sent (${code}), attempt ${attempts} of ${MAX_ATTEMPTS}`);
      }
```

7.7 Verification

The following commands were run on the final code on 2026-11-09.

Layer	How it runs	Result
Backend unit tests	`npx jest`	63 suites run (1 skipped), 791 tests passed, 2 skipped
Backend end-to-end tests	`jest --config ./test/jest-e2e.json --runInBand` against a real PostgreSQL database `stressforge_test`	33 suites, 429 tests passed
Lint and types	`eslint` and `tsc` on the changed files	Clean (one pre-existing type error in a test file of the admin dashboard, unrelated to this work)
Route access	Every route called without a token and with an ordinary user's token (Chapter 5)	115 routes; declared permissions and observed 403s agree
Deliberate breaks of the mailer	Four deliberate bugs introduced one at a time (replaying old notifications, ignoring the hourly cap, unlimited retries, mailing staff kinds)	All four were caught by the tests; the code was restored
Live email	A real SMTP send to a Mailtrap sandbox inbox	Delivered; no report text in the message (Figure 6.14)
Migrations	`pnpm migration:run` on an empty database	21 migrations applied; 25 tables including the bookkeeping table (Figure 6.15)

8. Conclusion and References

STRESSFORGE meets the guide's five minimum requirements and implements the bonus items listed in Section 5.3: middleware, custom decorators, an optional mailer, Swagger UI, an interceptor, environment configuration, migrations, a response envelope and advanced transactions. The audit log is partial (role changes and Support activity, not every create, update and delete). The system has 115 endpoints on 24 application tables, five roles enforced by a database-backed permission guard, and a simulation engine whose results are saved with the exact inputs that produced them.

Three differences from the guide are deliberate and stated: argon2 instead of bcrypt, Zod instead of class-validator, and HTTP 422 instead of 400 for validation errors. **Add the instructor's written acceptance here.**

Known limits

- The model is a mathematical estimate. It does not measure real systems and is not calibrated against load-test data from a deployed system.
- The optional AI features are off by default; the planned acceptance run against the hosted model has not been completed.
- Email is verified only against a sandbox inbox, and an email is sent only for Support replies and status changes.
- Not every create, update and delete is written to an audit log.
- The simulation admission queue is per API instance; running several instances would need a shared queue.
- The web client has no accessibility audit beyond keyboard support and contrast checks, and no translation.

Next steps

- A general audit interceptor for every create, update and delete.
- Failure injection (switch a component or region off) and a cost estimate per design.
- Calibration of the model against real load-test measurements.
- Support access to one frozen design revision, by the reporter's consent.

References

1. NestJS documentation. https://docs.nestjs.com
2. TypeORM documentation. https://typeorm.io
3. PostgreSQL documentation. https://www.postgresql.org/docs/
4. M. Jones, J. Bradley, N. Sakimura. JSON Web Token (JWT). RFC 7519. https://datatracker.ietf.org/doc/html/rfc7519
5. R. Fielding, M. Nottingham, J. Reschke. HTTP Semantics. RFC 9110. https://datatracker.ietf.org/doc/html/rfc9110
6. A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. Argon2 Memory-Hard Function for Password Hashing. RFC 9106. https://datatracker.ietf.org/doc/html/rfc9106
7. Zod documentation. https://zod.dev
8. OpenAPI Specification 3.1. https://spec.openapis.org/oas/v3.1.0
9. Next.js documentation. https://nextjs.org/docs
10. Bull job queue. https://github.com/OptimalBits/bull
11. Nodemailer. https://nodemailer.com
12. Mailtrap Email Sandbox. https://mailtrap.io